Deep Learning

Prof. André Gustavo Hochuli

gustavo.hochuli@pucpr.br

aghochuli@ppgia.pucpr.br

github.com/andrehochuli/teaching

Plano de Aula

- Apresentação do Professor
- Perceptron
- Redes Neurais Multicamadas
- Implementação em Python
- Discussões
  - Representação
  - Overffitting
  - Vanishing
  - Curse of Dimensionality

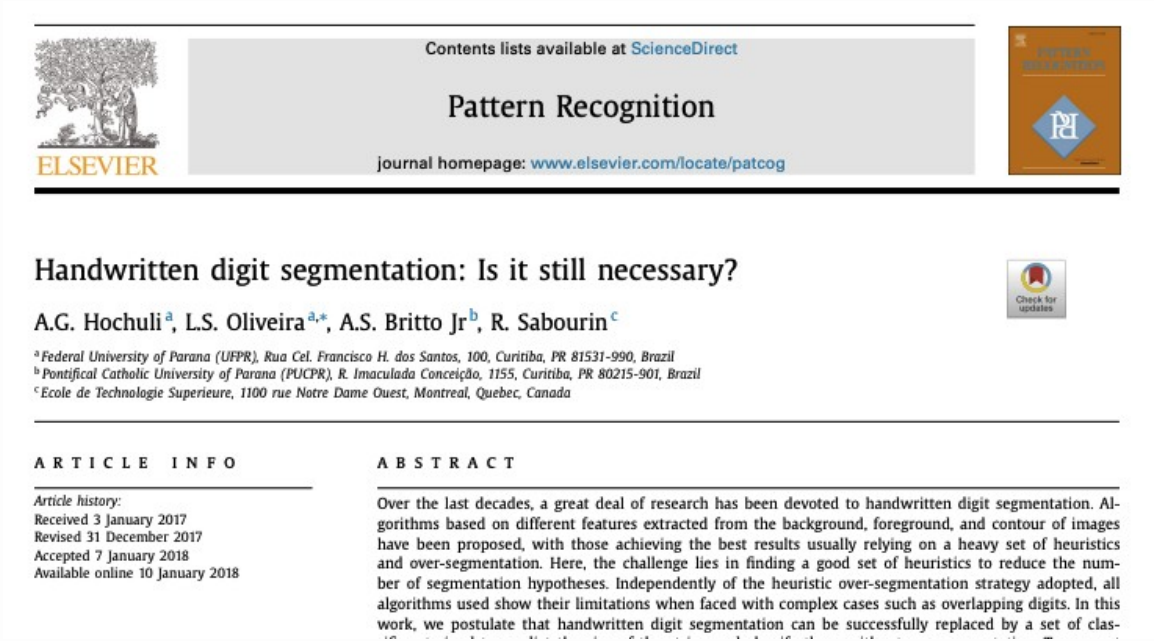
About me

- Andre G. Hochuli
- Researcher PPGIa/PUCPR
- Research topics
  - Computer Vision
  - Deep Learning
  - Pattern Recognition



- Contact Canvas
- gustavo.hochuli@pucpr.br
- https://www.linkedin.com/in/andre-hochuli-96117b18/
- https://scholar.google.com.br/citations?user=pbekYw4AAAAJ&hl=pt-BR

Handwritten Recognition



Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/patcog

Handwritten digit segmentation: Is it still necessary?

A.G. Hochuli [a], L.S. Oliveira [a,*], A.S. Britto Jr [b], R. Sabourin [c]

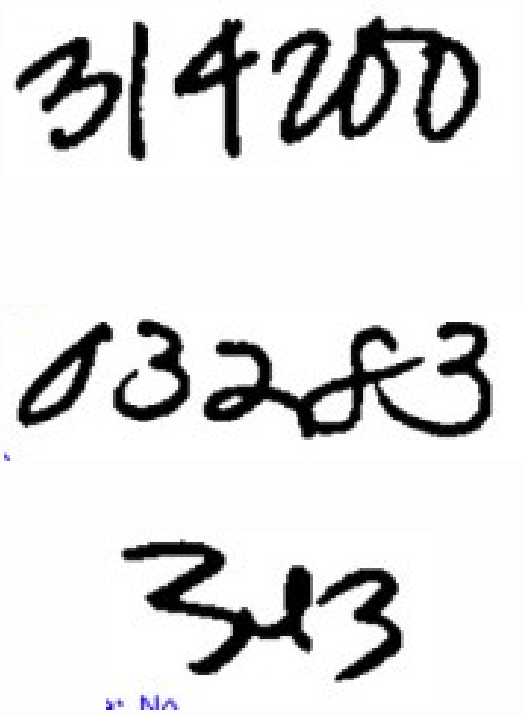
[a] Federal University of Parana (UFPR), Rua Cel. Francisco H. dos Santos, 100, Curitiba, PR 81531-990, Brazil
[b] Pontifical Catholic University of Parana (PUCPR), R. Imaculada Conceição, 1155, Curitiba, PR 80215-901, Brazil
[c] Ecole de Technologie Superieure, 1100 rue Notre Dame Ouest, Montreal, Quebec, Canada

ARTICLE INFO

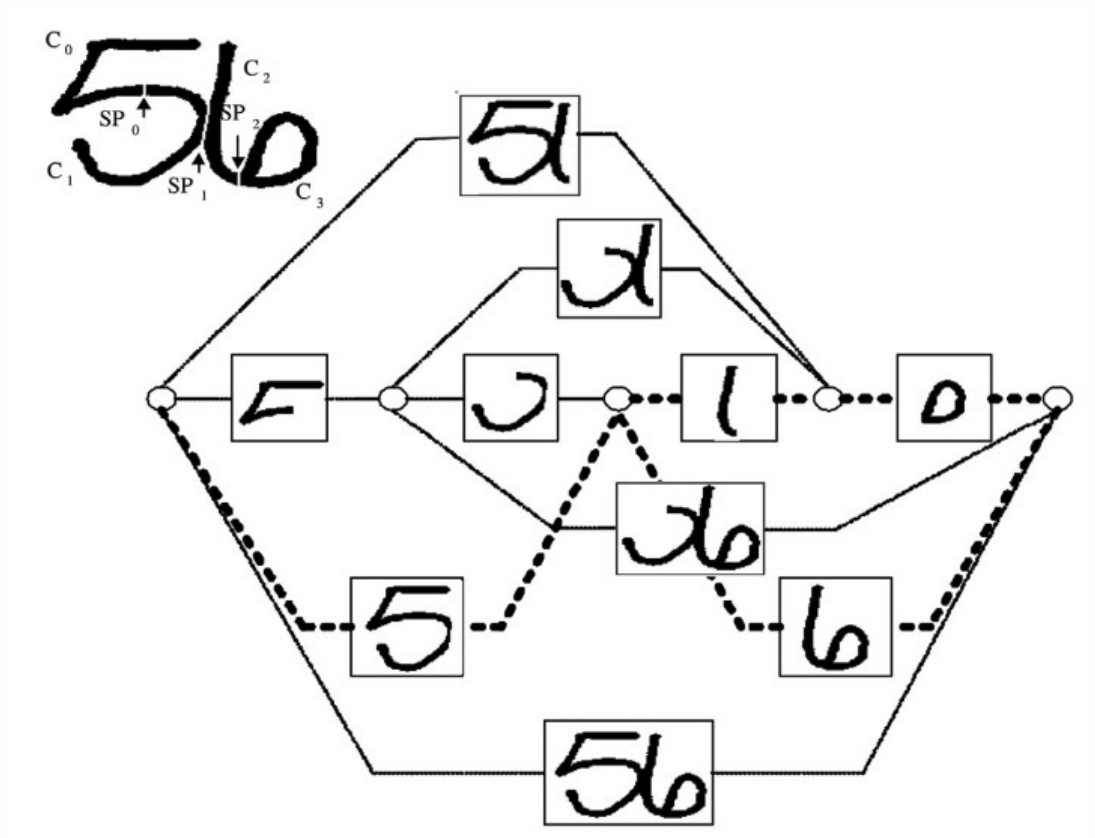
Article history:
Received 3 January 2017
Revised 31 December 2017
Accepted 7 January 2018
Available online 10 January 2018

ABSTRACT

Over the last decades, a great deal of research has been devoted to handwritten digit segmentation. Algorithms based on different features extracted from the background, foreground, and contour of images have been proposed, with those achieving the best results usually relying on a heavy set of heuristics and over-segmentation. Here, the challenge lies in finding a good set of heuristics to reduce the number of segmentation hypotheses. Independently of the heuristic over-segmentation strategy adopted, all algorithms used show their limitations when faced with complex cases such as overlapping digits. In this work, we postulate that handwritten digit segmentation can be successfully replaced by a set of clas-



Handwritten Recognition



Handwritten Recognition

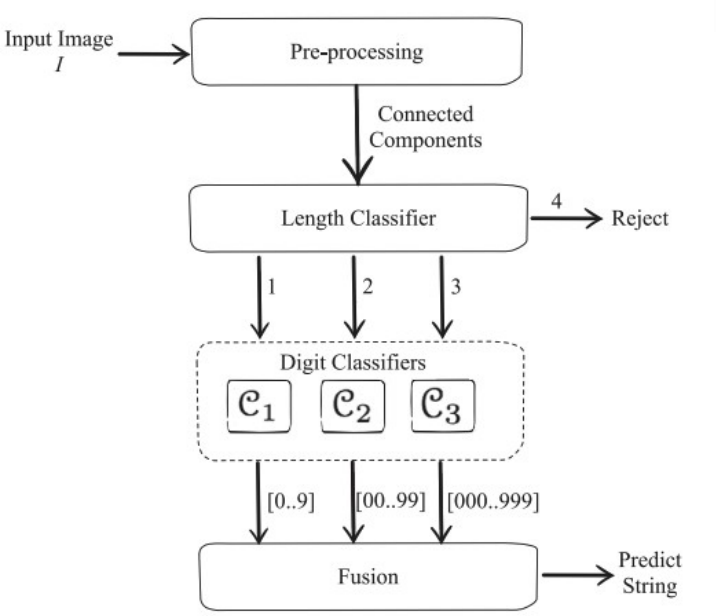
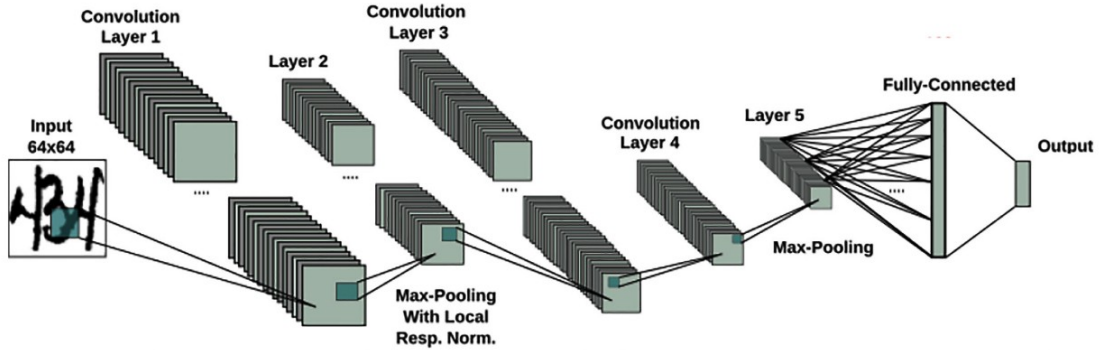


Fig. 1. Proposed framework.



Models	Input	Layer 1	Layer 2	Layer 3	Layer 4	Layer 5	Fully Connected	Output
1-Digit	64x64 Binary	Convolution 7x7@3@64	MaxPooling and Local Resp. Norm. 3x3@2@64	Convolution 3x3@1@128	Convolution 3x3@1@32	MaxPooling 3x3@2@32	128	10
2-Digit	64x64 Binary	Convolution 7x7@3@72	MaxPooling and Local Resp. Norm. 3x3@2@72	Convolution 3x3@1@192	Convolution 3x3@1@64	MaxPooling 3x3@2@64	1024	100
3-Digit	64x64 Binary	Convolution 7x7@1@24	MaxPooling and Local Resp. Norm. 2x2@2@24	Convolution 5x5@1@42	Convolution 5x5@1@32	MaxPooling 2x2@2@32	1200	1000

Handwritten Recognition

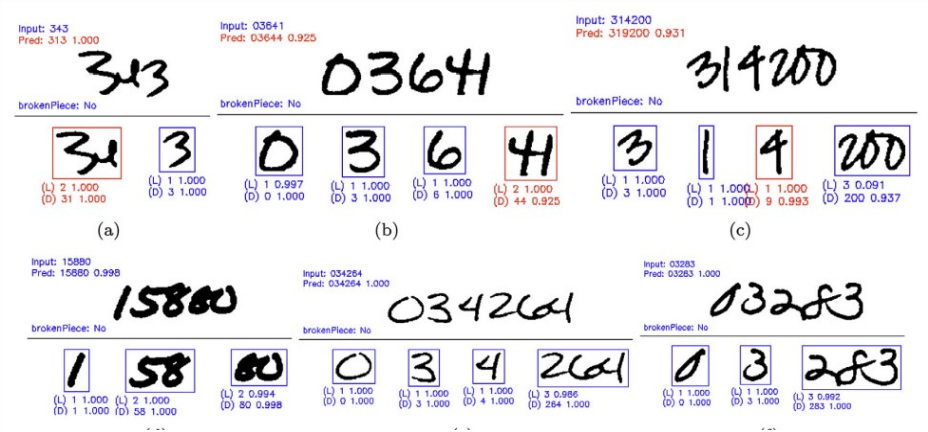


Table 7
Comparison of the recognition rates on NIST SD19.

Length	Samples	Britto et al. [2]	Oliveira et al. [19]	Oliveira et al. [18]	Sadri et al. [23]	* Sadri et al. [23]	Gattal et al. [8]	Proposed method	Samples	Liu et al. [16]	Ciresan et al. [4]
2	2370	94.8	96.8	97.6	95.5	98.9	99.0	97.6			
3	2385	91.6	95.3	96.2	91.4	97.2	97.3	96.2	1476	96.8	93.4
4	2345	91.3	93.3	94.2	91.0	96.1	96.5	94.6			
5	2316	88.3	92.4	94.0	88.0	95.8	95.9	94.1			
6	2169	89.0	93.1	93.8	88.6	96.1	96.6	93.3	1471	96.7	

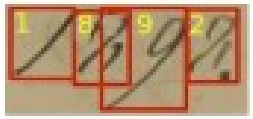
Handwritten Recognition

An End-to-End Approach for Recognition of Modern and Historical Handwritten Numeral Strings

Andre G. Hochuli, Alceu S. Britto Jr., Jean P. Barddal
Graduate Program in Informatics (PPGIa)
Pontifícia Universidade Católica do Paraná
Curitiba, PR - Brazil
{aghochuli, alceu, jean.barddal}@ppgia.pucpr.br

Luiz E. S. Oliveira
Department of Informatics (DInf)
Universidade Federal do Paraná
Curitiba, PR - Brazil
lesoliveira@inf.ufpr.br

Robert Sabourin
System Engineering Dept. (LIVIA)
École de Technologie Supérieure
Montreal, QC - Canada
robert.sabourin@etsmtl.ca



(a)

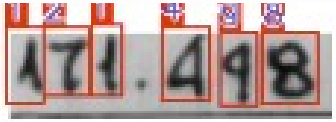


(b)



(c)





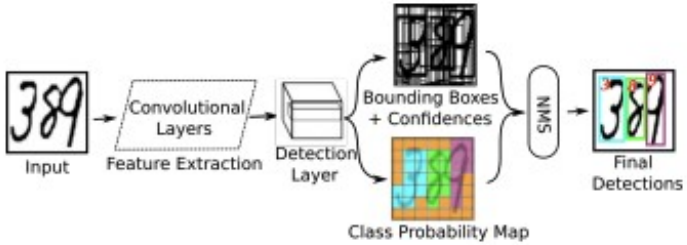


Table V
COMPARISON OF THE ACCURACY (%) ON ORAND-CAR DATASETS REPORTED BY [32]

Methods	CAR-A	CAR-B
Tebessa I[31]	37.05	26.62
Tebessa II[31]	39.72	27.72
Hochuli et al.[5]	50.10	40.20
Singapore[31]	52.30	59.30
Pernanbuco[31]	78.30	75.43
Beijing[31]	80.73	70.13
CRNN[33]	88.01	89.79
Saabni[34]*	85.80	
ResNet-RNN[35]	89.75	91.14
ResNet-RNN[32]	91.89	93.79
YoLo	**96.20**	**96.80**

* Unified CAR-A and CAR-B datasets

Table VI
BENCHMARK FOR ARDIS DATASET IV (SINGLE DIGITS) OF MODELS ON DIFFERENT TRAINNING PROTOCOLS (REPORTED BY [1])

Method	Accuracy (%)			
	Case I	Case II	Case III	Case IV
YoLo	**87.60**	**64.10**	**99.70**	**99.27**
Hochuli et al.[5]	67.20	51.90	83.30	60.55
CNN	58.80	35.44	98.60	99.34
HOG-SVM	56.18	33.18	95.50	98.08
RNN	45.74	28.96	91.12	96.74
kNN	50.15	22.72	89.60	96.63
SVM	43.40	30.62	92.40	96.48
Random Forest	20.12	17.15	87.00	93.12

Parking Lot Monitoring

Evaluation of Different Annotation Strategies for Deployment of Parking Spaces Classification Systems

Andre G. Hochuli, Alceu S. Britto Jr.
Graduate Program in Informatics
Pontifícia Universidade Católica do Paraná
Curitiba, PR - Brazil
{aghochuli, alceu}@ppgia.pucpr.br

Paulo R. L. de Almeida
Department of Informatics
Universidade Federal do Paraná
Curitiba, PR - Brazil
paulorla@ufpr.br

Williams B. S. Alves, Fábio M. C. Cagni
Pontifícia Universidade Católica do Paraná
Curitiba, PR - Brazil
{williams.alves, fabio.cagni}@pucpr.edu.br

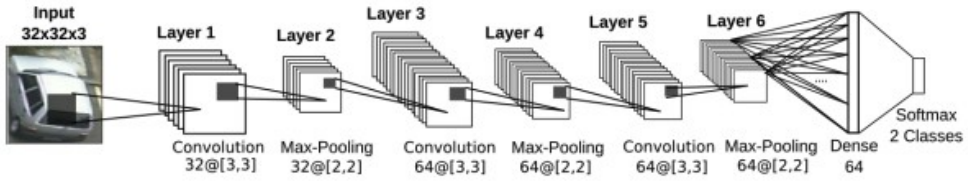


Fig. 1. The 3-convolutional layers architecture used to classify the parking spots.



(a) UFPR04 (b) UFPR05

(c) PUCPR

Fig. 2. The PKLOT dataset comprises three scenarios named a) UFPR04, b) UFPR05 and c) PUCPR.



(a) Polygon (b) Bounding Box (c) Fixed Square

Fig. 3. Types of parking spots location demarcations.

Parking Lot Monitoring

TABLE III

ACCURACIES (%) OF MODELS TRAINED WITH DIFFERENT ANNOTATION TYPES OF PKLOT DATASET AND TESTED IN A CROSS-DOMAIN STRATEGY

Polygon

	UFPR04 Test Set		UFPR05 Test Set		PUCPR Test Set	
Models	**W/O Aug**	**W/ Aug**	**W/O Aug**	**W/ Aug**	**W/O Aug**	**W/ Aug**
UFPR04	98,5	98,2	92,1	89,1	98,4	97,4
UFPR05	91,4	89,4	99,5	99,5	91,8	90,7
PUCPR	98,9	98,0	92,0	90,4	99,8	99,8

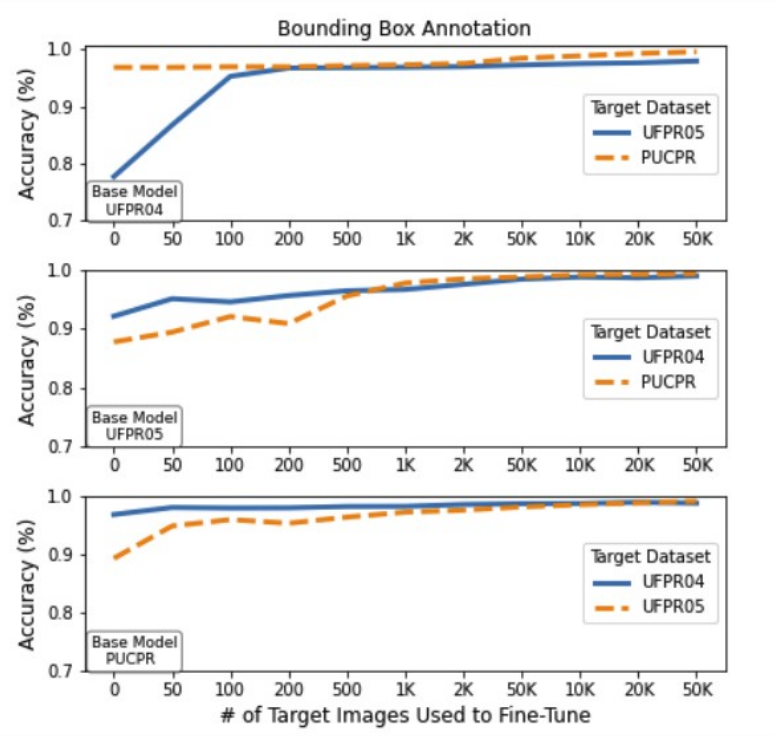
Bounding Box

	UFPR04 Test Set		UFPR05 Test Set		PUCPR Test Set	
Models	**W/O Aug**	**W/ Aug**	**W/O Aug**	**W/ Aug**	**W/O Aug**	**W/ Aug**
UFPR04	98,6	99,1	77,7	74,6	96,7	96,7
UFPR05	92,1	92,5	99,5	99,6	87,7	89,0
PUCPR	96,7	96,7	89,2	90,4	99,8	99,8

Fixed Square

	UFPR04 Test Set		UFPR05 Test Set		PUCPR Test Set	
Models	**W/O Aug**	**W/ Aug**	**W/O Aug**	**W/ Aug**	**W/O Aug**	**W/ Aug**
UFPR04	97,4	93,7	60,5	71,0	89,2	71,0
UFPR05	83,4	89,8	99,3	98,9	87,0	91,3
PUCPR	94,7	95,4	91,8	93,2	99,7	99,8

*With (W/) or Without (W/O) Data Augmentation (Aug)



Parking Lot Monitoring

Optimizing Parking Space Classification: Distilling Ensembles into Lightweight Classifiers

Paulo Luza Alves*, André Hochuli†, Luiz Eduardo de Oliveira*, Paulo Lisboa de Almeida*
* Departamento de Informática (DInf), Universidade Federal do Paraná, Curitiba, PR - Brazil
{paulomateus,luiz.oliveira,paulorla}@ufpr.br
† Programa de Pós-Graduação em Informática (PPGIa), Pontifícia Universidade Católica do Paraná, Curitiba, PR - Brazil
aghochuli@ppgia.pucpr.br

***Abstract*—When deploying large-scale machine learning models for smart city applications, such as image-based parking lot monitoring, data often must be sent to a central server to perform classification tasks. This is challenging for the city's infrastructure, where image-based applications require transmitting large volumes of data, necessitating complex network and hardware infrastructures to process the data. To address this issue in image-based parking space classification, we propose creating a robust ensemble of classifiers to serve as Teacher models. These Teacher models are distilled into lightweight and specialized Student models that can be deployed directly on edge devices. The knowledge is distilled to the Student models through pseudo-labeled samples generated by the Teacher model, which are utilized to fine-tune the Student models on the target scenario. Our results show that the Student models, with 26 times fewer parameters than the Teacher models, achieved an average accuracy of 96.6% on the target test datasets, surpassing the Teacher models, which attained an average accuracy of 95.3%.**

I. Introduction

When deploying machine learning models that deal with



Fig. 1. Occupied (red) and empty (blue) parking spaces – PKLot [1].

small-scale deployment with 1,000 cameras. Suppose each camera sends a 1280 × 720 pixels JPEG compressed image to a central server every 30 seconds. In that case, it will result in a transfer of 35 gigabytes of data per hour to the server, not accounting for the network's overheads (estimated using the PKLot dataset [1]). This data transfer can approximately

Deep Single Models vs. Ensembles: Insights for a Fast Deployment of Parking Monitoring Systems

Andre Gustavo Hochuli*
Jean Paul Barddal
Graduate Program in Informatics (PPGIa)
Pontifícia Universidade Católica do Paraná
Curitiba, PR - Brazil
{aghochuli, jpbarddal}@ppgia.pucpr.br

Gillian Cezar Palhano
Leonardo Matheus Mendes
Pontifícia Universidade Católica do Paraná
Curitiba, PR - Brazil
gillian.palhano@pucpr.edu.br
l.mendes2@pucpr.edu.br

Paulo Ricardo Lisboa de Almeida
Department of Informatics (DInf)
Universidade Federal do Paraná
Curitiba, PR - Brazil
paulorla@ufpr.br

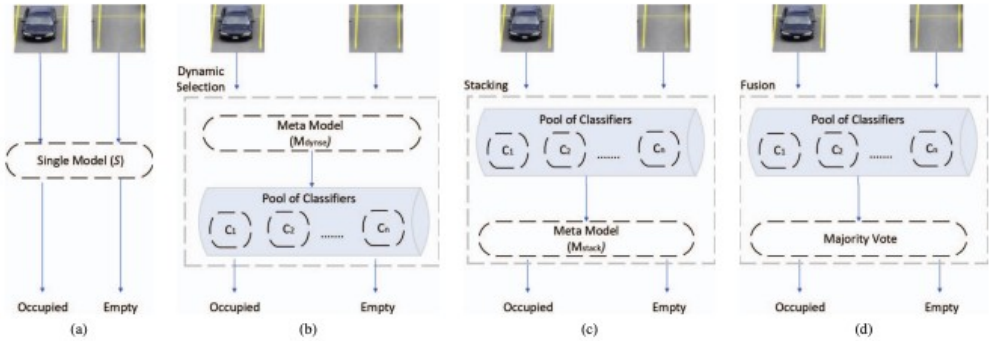


Fig. 1. The proposed strategies assessed for parking lot monitoring systems in cross-dataset scenarios: (a) a single model, and the ensemble-based frameworks named (b) dynamic selection and (c) stacking, and finally (d) the majority vote fusion-based framework.

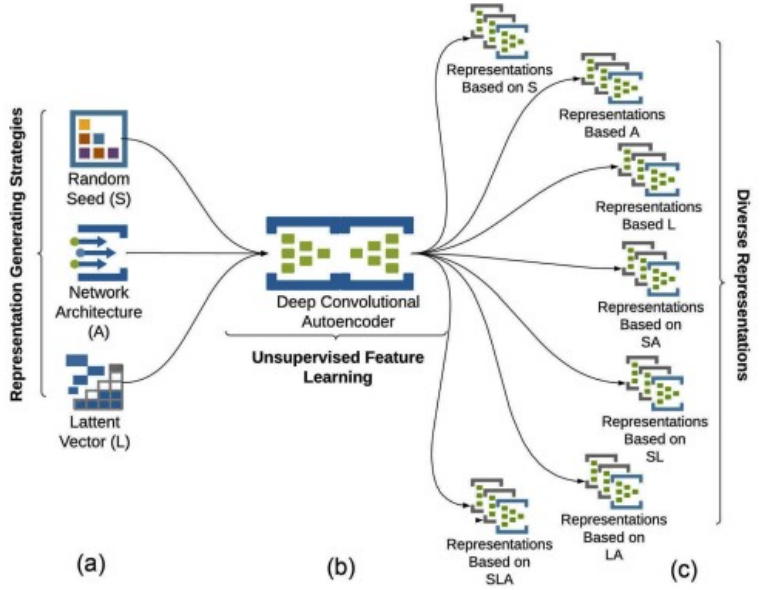
Facial Expression

ORIGINAL ARTICLE

Representation ensemble learning applied to facial expression recognition

Bruna Rossetto Delazeri[1] · Andre Gustavo Hochuli[1] · Jean Paul Barddal[1] · Alessandro Lameiras Koerich[2] · Alceu de Souza Britto Jr.[1,3]

Received: 2 February 2024 / Accepted: 3 October 2024 / Published online: 18 November 2024
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2024



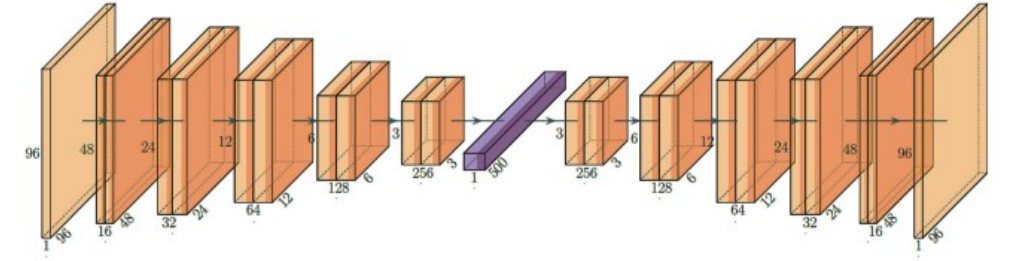
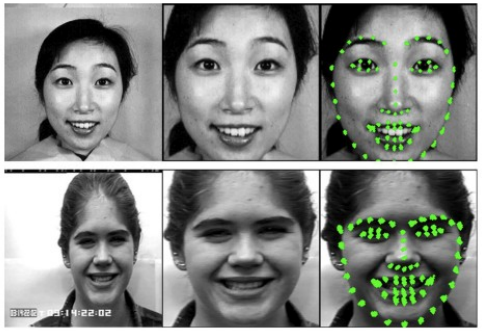


Fig. 3 Default CAE Architecture



Audio Replay Attack

~~A Dissimilarity-Based~~ Countermeasure for Detecting Replay Attacks in Speaker Verification

Maria Eduarda Maciel Pinto
Programa de Pós-Graduação em Informática (PPGIa)
Pontifícia Universidade Católica do Paraná
Curitiba, State of Paraná, Brazil
Email: mariamaciel@ppgia.pucpr.br

Alceu de Souza Britto Jr., Andre Gustavo Hochuli
Programa de Pós-Graduação em Informática (PPGIa)
Pontifícia Universidade Católica do Paraná
Curitiba, State of Paraná, Brazil
Email: {alceu, aghochuli}@ppgia.pucpr.br



Fig. 4: Proposed Pipeline: a deep architecture performs feature extraction from a MFCC representation, serving as input to the dissimilarity domain.

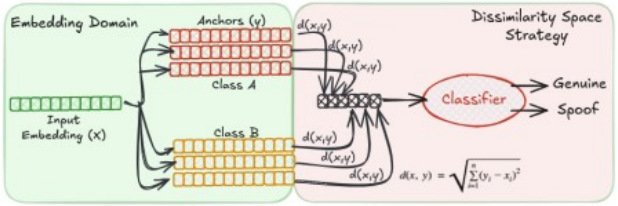


Fig. 2: The Dissimilarity Space strategy transforms the embedding (x) by generating a K-dimensional dissimilarity representation, consisting of the Euclidean distances to all anchors ($y_0, ..., y_K$).

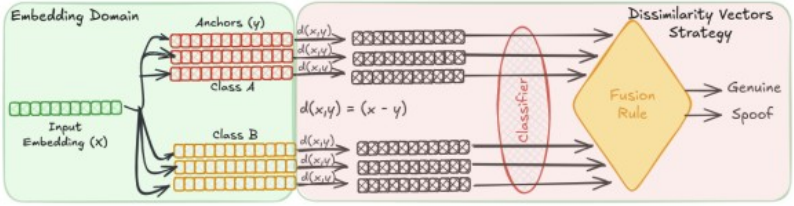
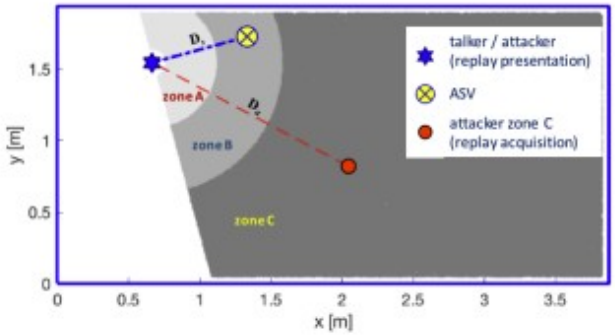


Fig. 3: The Dissimilarity Vectors strategy transforms the embedding (x) into K new representations by calculating the element-wise differences between the input and each of the anchors ($y_0, ..., y_K$).

A. ASVSpoof 2019 Dataset



Plant Species Recognition

Combining Muti-Layer Features For Plant Species Classification in a Siamese Network

Matheus Moresco[1], Alceu de S. Britto Jr[1,2], Yandre M. G. Costa[3], Luciano J. Senger[1], Andre G. Hochuli[2]

[1]State University of Ponta Grossa (UEPG), Ponta Grossa (PR), Brazil
Email: {3100120007015, ljsenger, alceubritto}@uepg.br
[2]Graduate Program in Informatics (PPGIa), Pontifícia Universidade Católica do Paraná (PUCPR), Curitiba (PR), Brazil
Email: {aghochuli, alceu}@ppgia.pucpr.br
[3]Department of Informatics (DIN), Universidade Estadual de Maringá (UEM), Maringá (PR), Brazil
Email: {yandre}@din.uem.br

TABLE I
CNN AND SNN PERFORMANCE RATES (%) IN FLAVIA AND MALAYAKEW DATASETS

Architetures	Datasets			
	Flavia		MalayaKew	
	CNN	SNN	CNN	SNN
VGG-16	97.9	97.7	90.9	70.8
MobileNet	**99.5**	98.8	**96.5**	**84.5**
DenseNet201	98.6	**99.4**	95.6	82.9
ResNet50	92.1	74.2	50.2	28.4
Inception v3	98.2	95.7	87.9	66.8

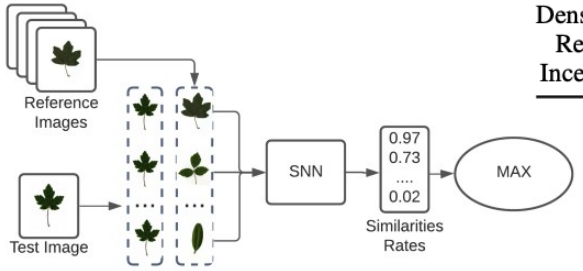


TABLE II
PERFORMANCE (%) OF THE SNN WITH MULTI-LAYER FEATURES IN FLAVIA DATASET

VGG-16 Architecture

Intermediate Layers	Entire Vector	Standard (256)	
	Concatenate	*Concatenate*	*Add*
FC Layer Only	97.3	-	-
B5	92.8	95.9	93.3
B5+B4	96.3	98.4	97.0
B5+B4+B3	97.0	**98.6**	98.2
B5+B4+B3+B2	98.4	98.4	98.2
B5+B4+B3+B2+B1	97.0	97.2	97.9

MobileNet Architecture

Intermediate Layers	Entire Vector	Standard (256)	
	Concatenate	*Concatenate*	*Add*
FC Layer Only	98.7	-	-
B5	99.3	97.2	99.3
B5+B4	98.6	98.9	98.4
B5+B4+B3	98.2	99.4	99.4
B2+B3+B4+B5	98.0	99.3	**99.8**
B1+B2+B3+B4+B5	98.7	98.7	98.2

DenseNet Architecture

Intermediate Layers	Entire Vector	Standard (256)	
	Concatenate	*Concatenate*	*Add*
FC Layer Only	99.4	-	-
B5	98.0	99.1	99.5
B5+B4	98.7	99.4	99.8
B5+B4+B3	98.9	99.3	**100.0**
B2+B3+B4+B5	99.1	99.3	99.4
B1+B2+B3+B4+B5	99.1	99.8	99.8

Research Interests

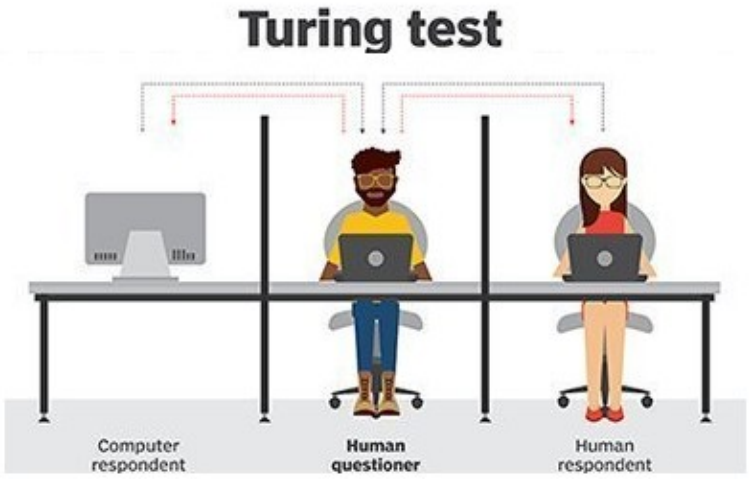
- Medical Image Analysis
- Edge AI
- Federated Learning
- Mixture of Experts
- Domain Adaptation

Aprendizagem de Máquina
Introdução e Conceitos

Inteligência Artificial

"Tópico da Ciência da Computação cujo objetivo é desenvolver **soluções tecnológicas que simulam o raciocínio humano**, ou seja, a inteligência que é característica dos seres humanos."

"Máquinas podem pensar?" - **Alan Turing** (1950)



IA passa a ser considerada ciência em 1956 (Dartmouth College, USA).

Inteligência Artificial

A inteligência artificial já faz parte do nosso cotidiano há algum tempo.

- Filtros de SPAM
- Sistemas de Recomendação
- Reconhecimento de Padrões
  - Reconhecimento Facial em Imagens
  - Biometria Ocular e Impressão Digital
  - Análise e Monitoramento de Video
  - Inspeção Industrial

Inteligência Artificial

Mais recentemente, dado os modelos profundos, o uso de I.A se tornou mais evidente.





Inteligência Artificial

Mais recentemente, dado os modelos profundos, o uso de I.A se tornou mais evidente.



AI is acquiring a sense of smell that can detect illnesses in human breath

Computers can take minutes to autonomously analyse a sample that previously took hours to test

Andrea Soltoggio | Wednesday 13 June 2018 12:57 | 0 comments

Curti Click to follow The Independent



Artificial intelligence that can teach? It's already happening

Share on Facebook Share on Twitter

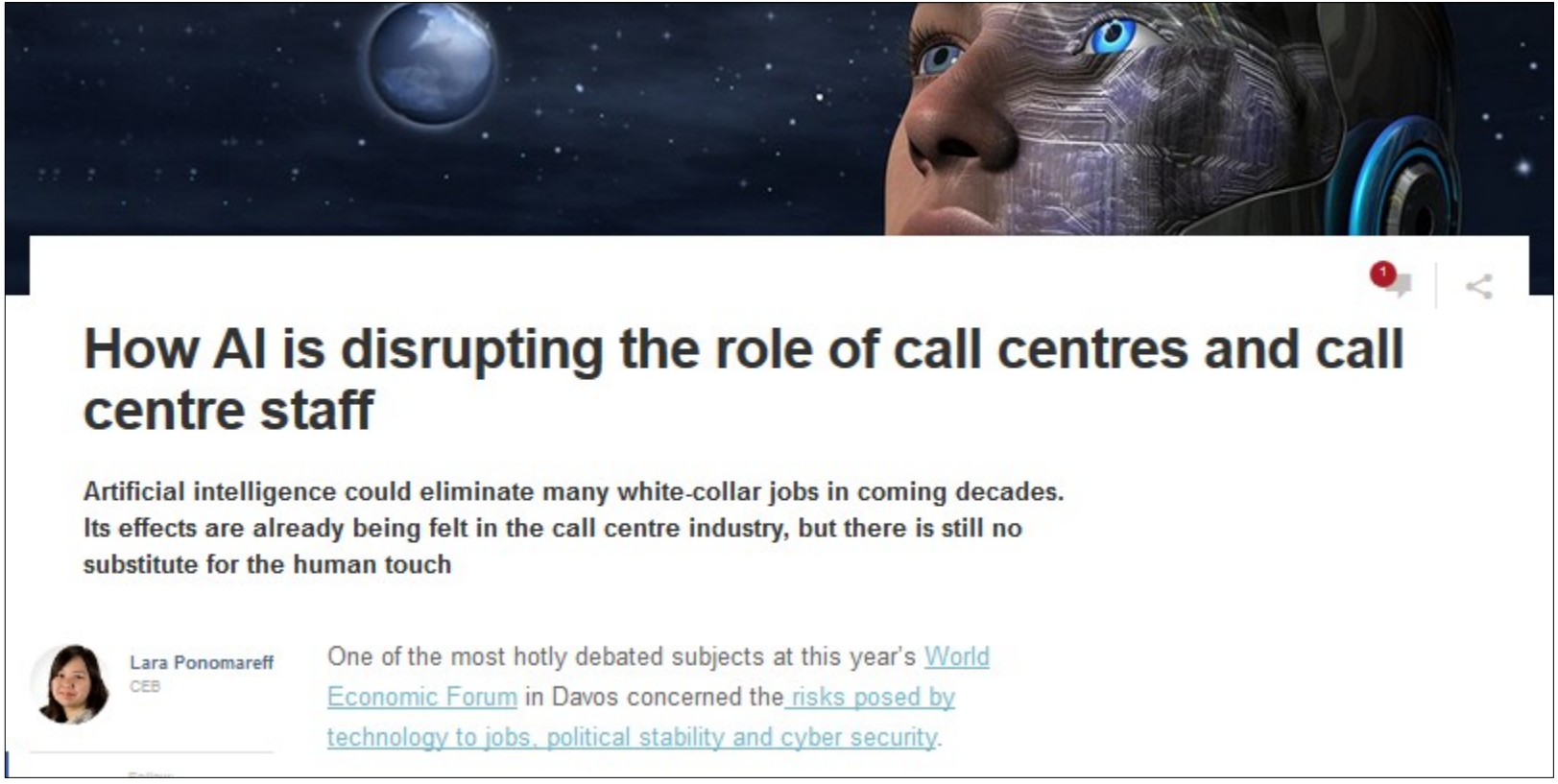
ABC Science By Carl Smith for the Science Show

Updated 15 June 2018 at 10:10 pm

First posted 15 June 2018 at 5:04 pm

Inteligência Artificial

Mais recentemente, dado os modelos profundos, o uso de I.A se tornou mais evidente.



How AI is disrupting the role of call centres and call centre staff

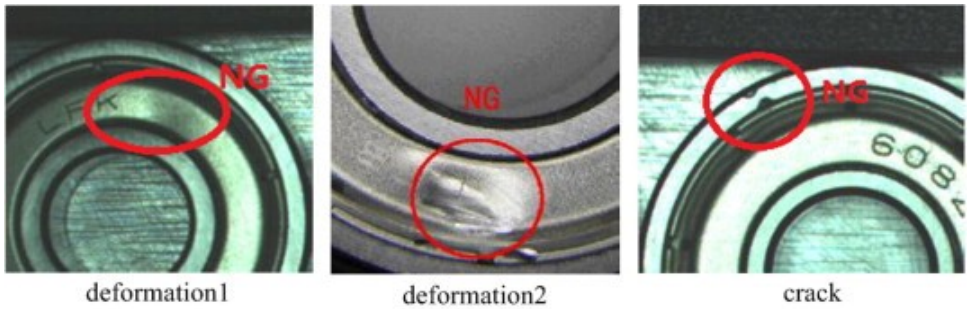
Artificial intelligence could eliminate many white-collar jobs in coming decades. Its effects are already being felt in the call centre industry, but there is still no substitute for the human touch

Lara Ponomareff
CEB

One of the most hotly debated subjects at this year's World Economic Forum in Davos concerned the risks posed by technology to jobs, political stability and cyber security.

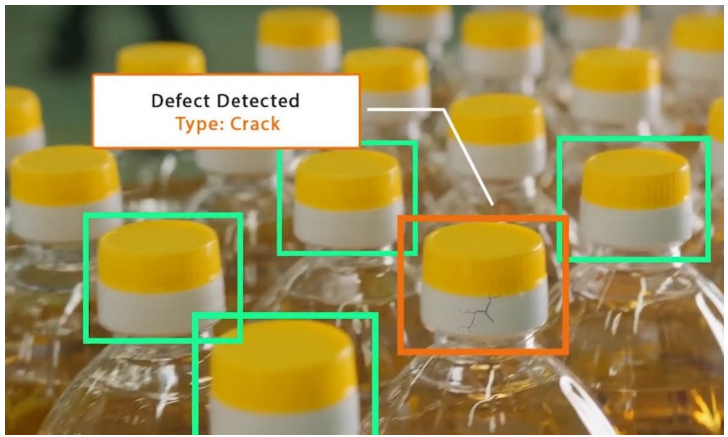
Inteligência Artificial

Mais recentemente, dado os modelos profundos, o uso de I.A se tornou mais evidente.



deformation1 deformation2 crack

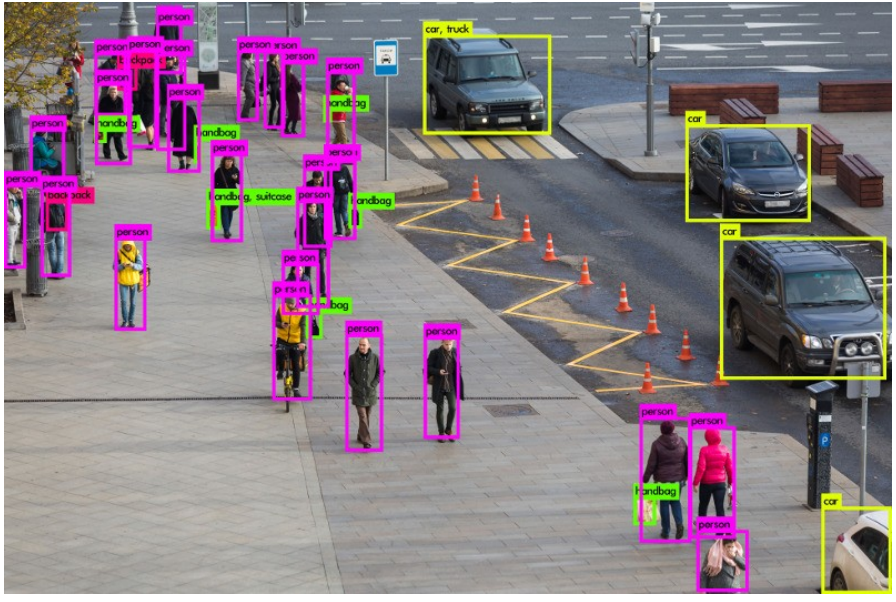
scratch dust

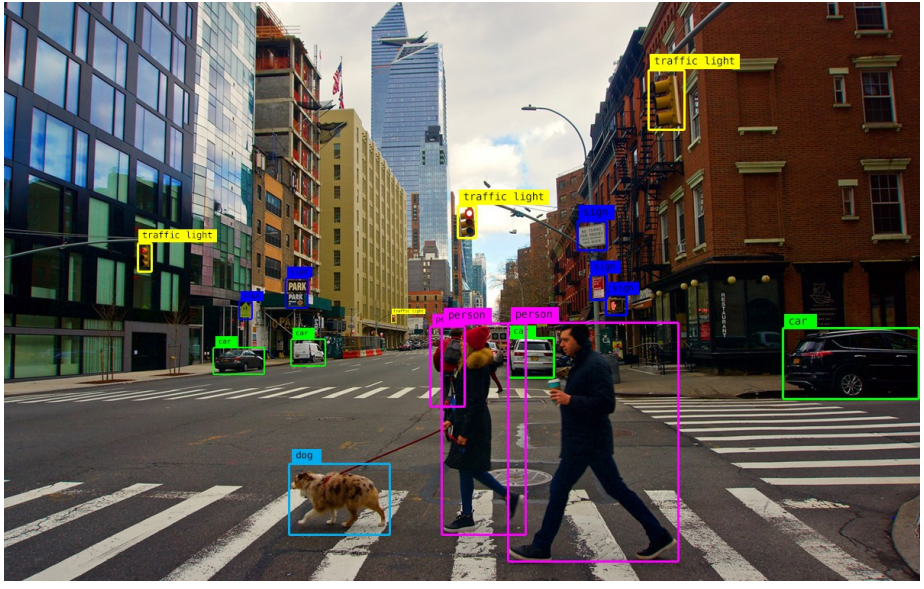




Inteligência Artificial

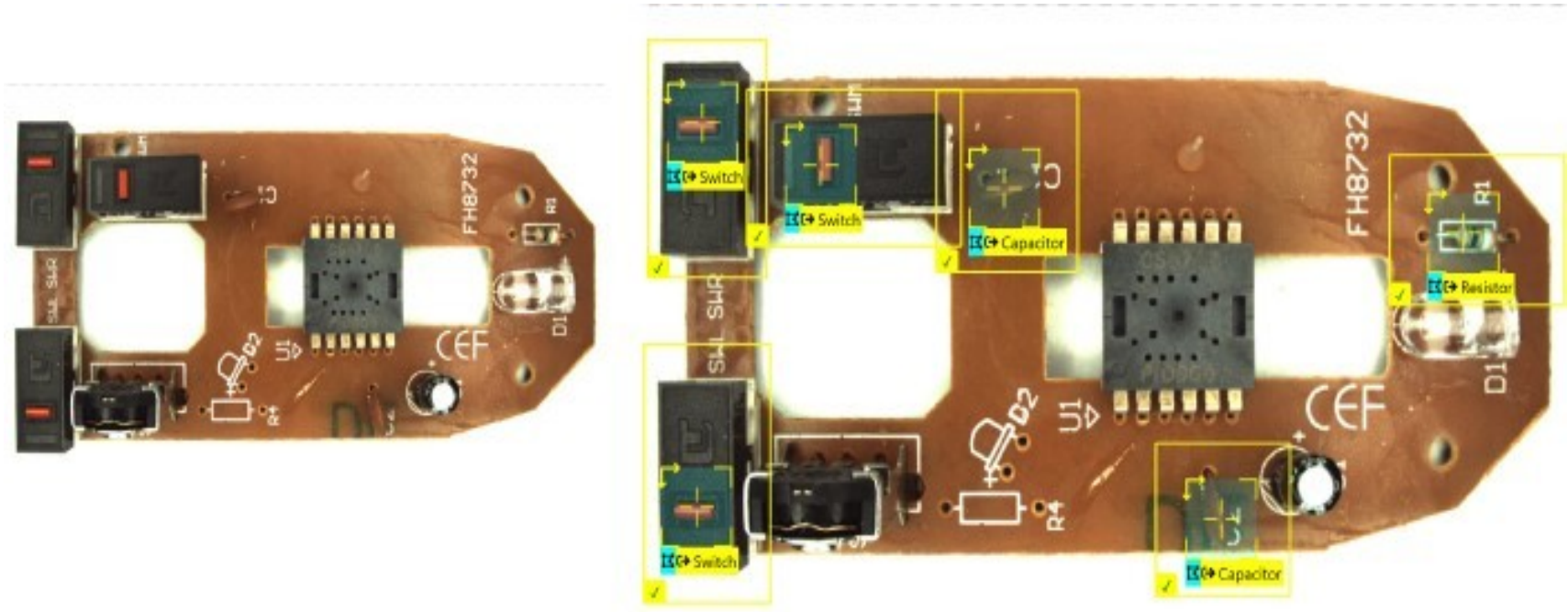
Mais recentemente, dado os modelos profundos, o uso de I.A se tornou mais evidente.





Inteligência Artificial

Mais recentemente, dado os modelos profundos, o uso de I.A se tornou mais evidente.



Inteligência Artificial

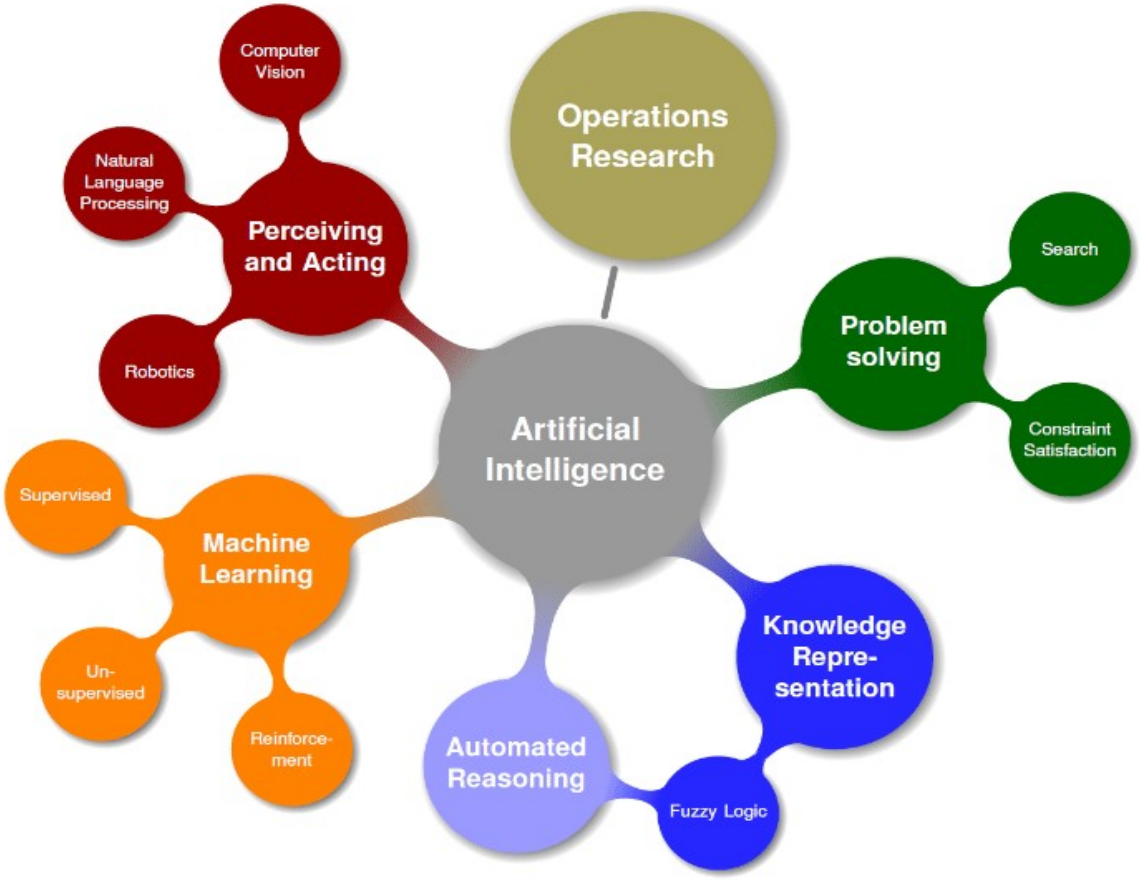
Mais recentemente, dado os modelos profundos, o uso de I.A se tornou mais evidente.



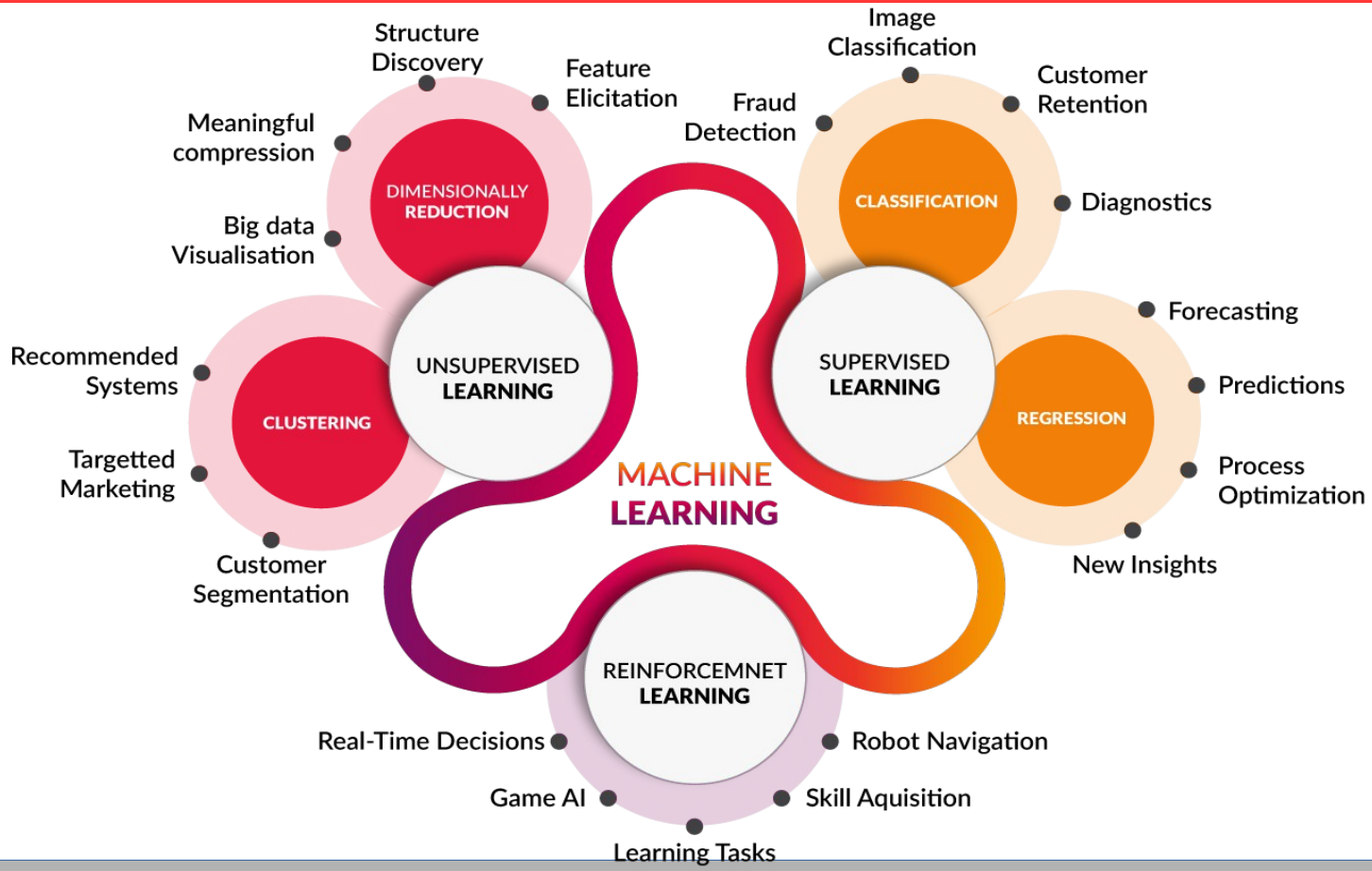




Inteligência Artificial

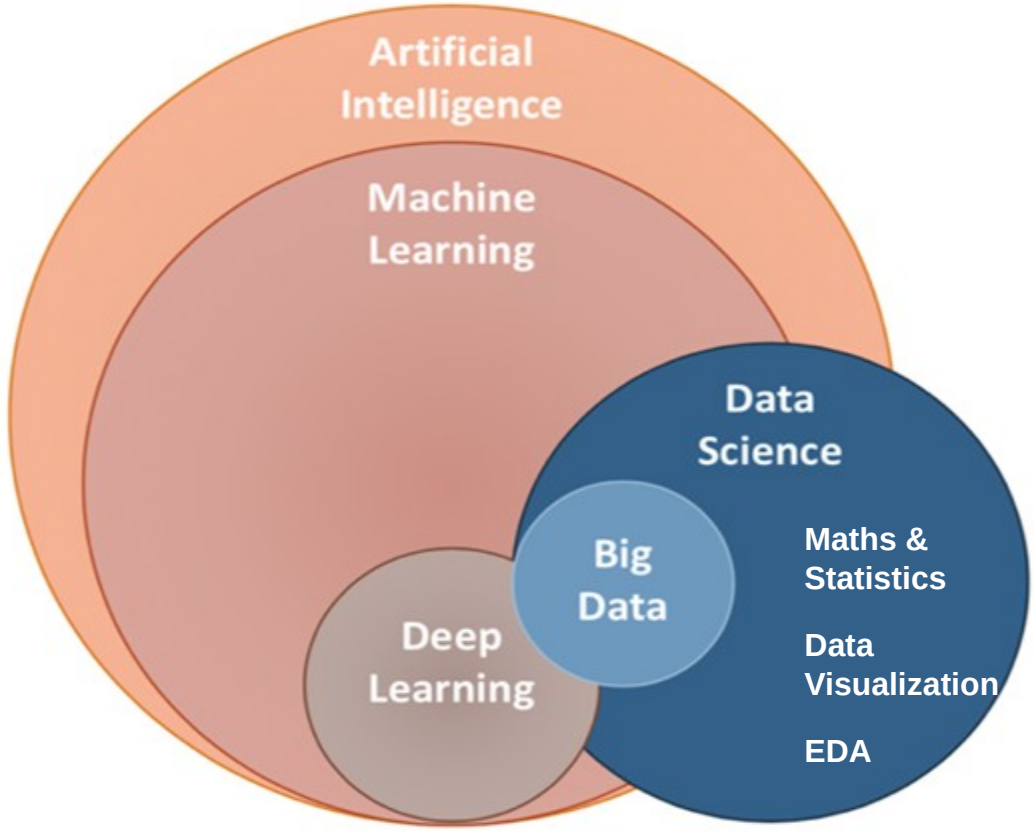


Aprendizado de Máquina



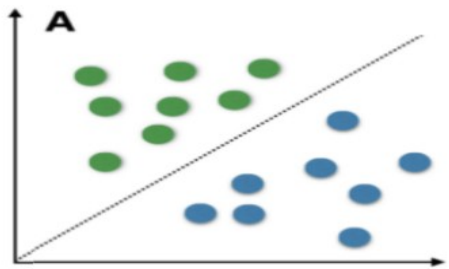
Aprendizado de Máquina

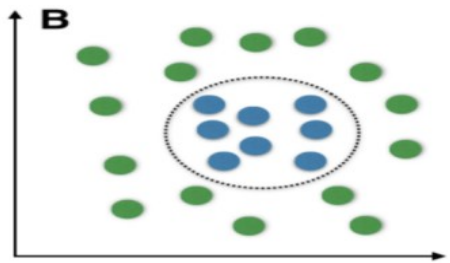
Temas Emergentes

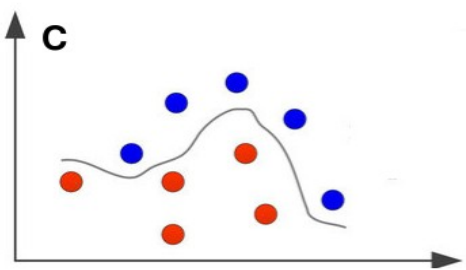


Quando aplicar?

- Representação do problema não é linearmente separável
- A solução não é determinística
- Complexidade Alta (Multi-Dimensões)



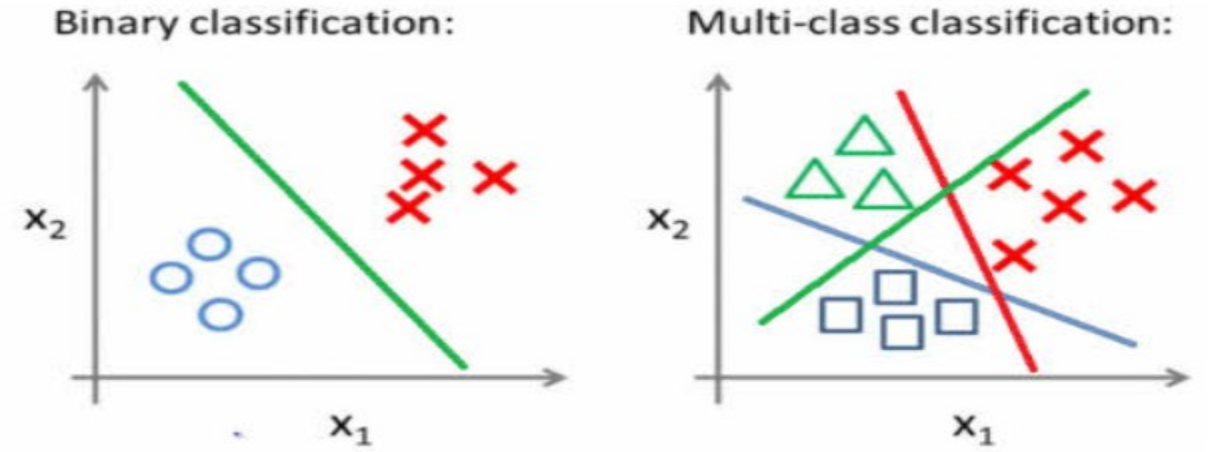




Redes Neurais

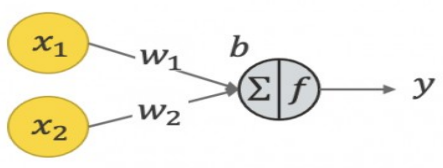
Discussões Iniciais

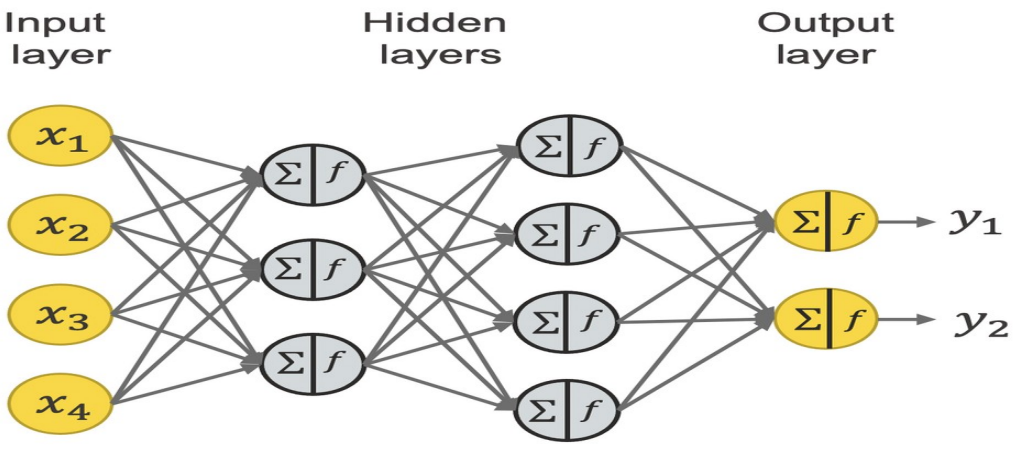
- Classificação Binária vs Multi-Classes
  - One vs All
  - One vs One



Redes Neurais Artificiais

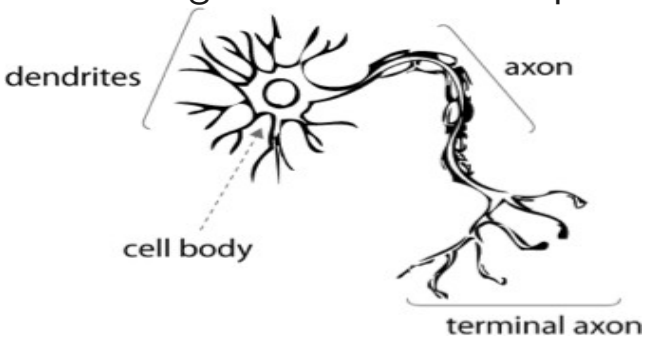
- Resolve problemas complexos
- Arquitetura é composta de neurônios artificiais conectados
- Número de Camadas é ilimitado



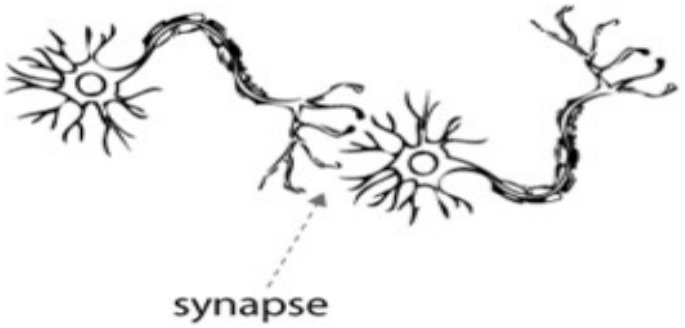


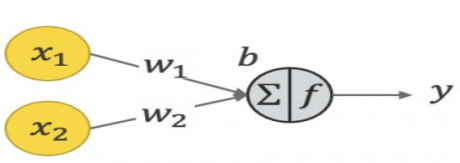
Redes Neurais Artificiais

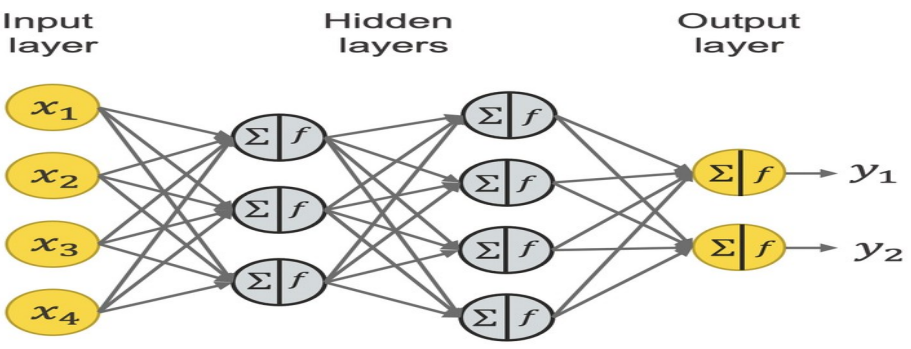
- Modelo Biológico vs Modelo Computacional





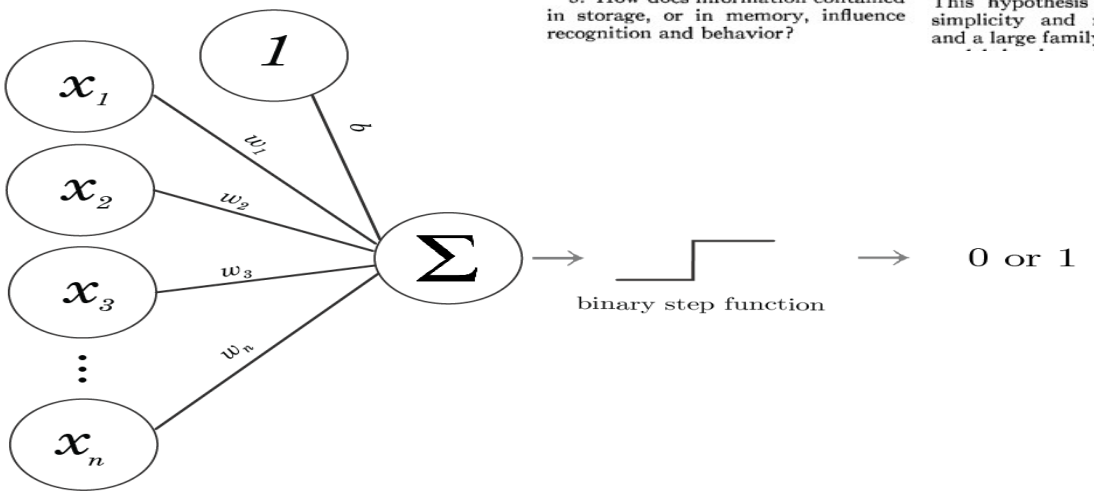






Perceptron

- Neuronio Artificial (Rosenblatt) (1958)
- Classificador Binário (0 | 1)
  - Pesos
  - Bias
  - Função de Ativação



Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN[1]

F. ROSENBLATT

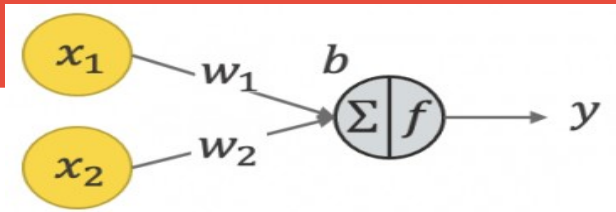
Cornell Aeronautical Laboratory

If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental questions:

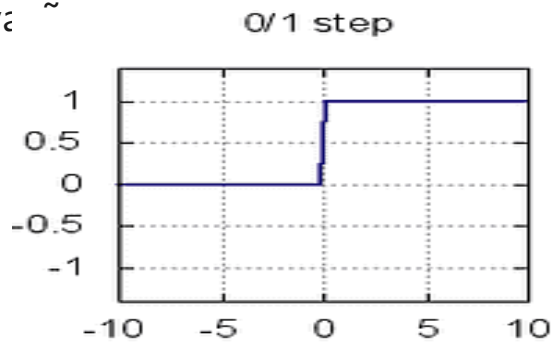
1. How is information about the physical world sensed, or detected, by the biological system?
2. In what form is information stored, or remembered?
3. How does information contained in storage, or in memory, influence recognition and behavior?

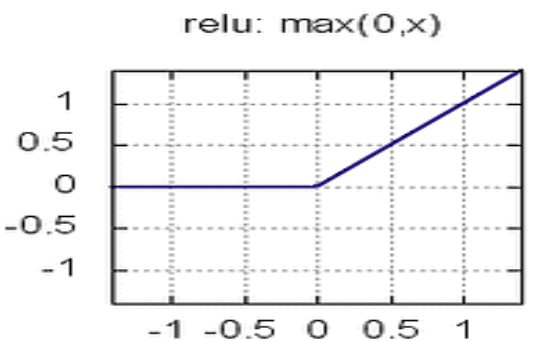
and the stored pattern. According to this hypothesis, if one understood the code or "wiring diagram" of the nervous system, one should, in principle, be able to discover exactly what an organism remembers by reconstructing the original sensory patterns from the "memory traces" which they have left, much as we might develop a photographic negative, or translate the pattern of electrical charges in the "memory" of a digital computer. This hypothesis is appealing in its simplicity and ready intelligibility, and a large family of theoretical brain

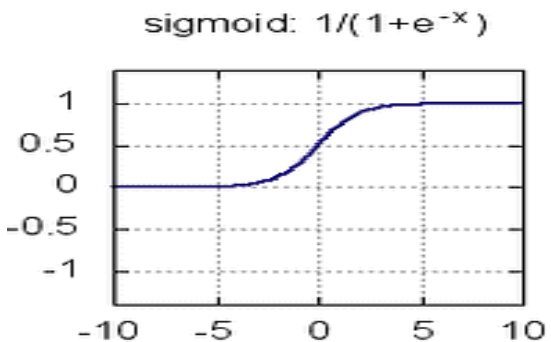
Perceptron

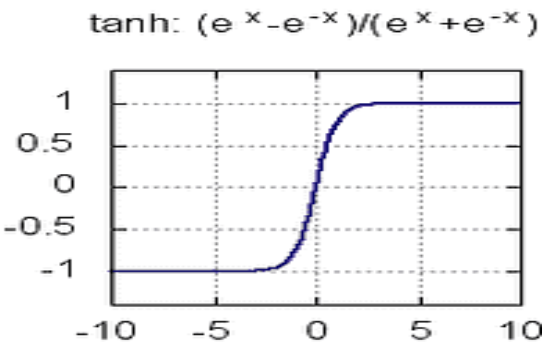


- Função de Ativa ˜



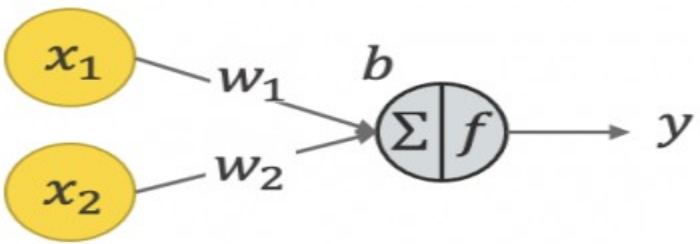




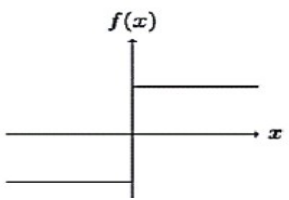


Perceptron

- Va $x_1 * w_1 + x_2 * w_2 + b$ da com 2 atributos (features)
  -



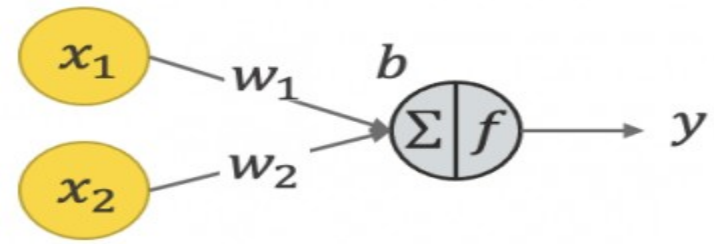
- Considere:
  - x1 = 1
  - x2 = 0
  - w1 = 0.2
  - w2 = 0.8
  - b = 0
  - Y = 1
  - Ativação: Step Function



Perceptron

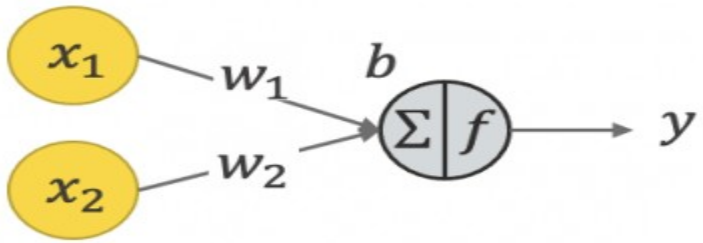
- Treinamento
  - Atualizar w1, w2 e b
  - Até que seja atingido o ‘y’ desejado

- Como?
  - Loss (Função de Perda)
  - Learning Rate (Atualização dos pesos)



Perceptron

- Rotina de Treino
  - Inicializar pesos e bias aleatórios (adam , xavier)
  - Definir a função de ativação (exemplo: STEP Function)
  - Para cada exemplo no conjunto de treinamento:
    - a. Calcular a saída do Perceptron
    - b. Atualizar os pesos e bias com base no erro



- Rotina de Teste
  - Computar com os pesos do treino

Perceptron – Implementação Python

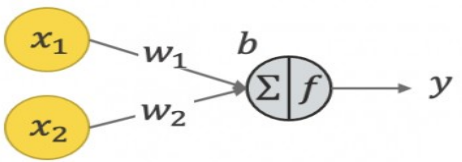
- Definição das funções básicas

```python
import numpy as np

def activation(x):
  return

def predict(X,weights,bias):
  return

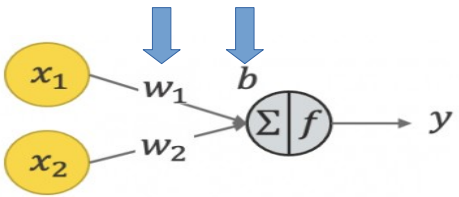
def fit(X, y, learning_rate=0.001, epochs=100):
  return
```



Perceptron – Implementação Python

- Fit()
  -
```
def fit(X, y, learning_rate=0.001, epochs=100):
      n_features = X.shape[1]

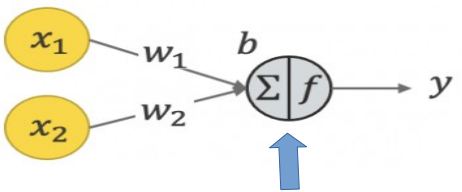
      # Iniacialização
      weights = weights = np.random.rand(n_features)
      bias = 0
```
  -



Perceptron – Implementação Python

- Fit()
  - Iteração sobre a base
    - Computa o produto escalar dos pesos e bias
    - Função de Ativação

```
def activation(x):
  return np.where(x >= 0, 1, 0)
```




```
def fit(X, y, learning_rate=0.001, epochs=100):
      n_features = X.shape[1]

      # Iniacialização
      weights = weights = np.random.rand(n_features)
      bias = 0

      # Iterating until the number of epochs
      for epoch in range(epochs):
          # iteração entre as amostras
          for i in range(len(X)):
              z = np.dot(X, weights) + bias # Produto escalar e bias
              y_pred = activation(z) #Função de ativação
              loss = (y[i] - y_pred[i]) #calculo da perda
```

Perceptron – Implementação Python

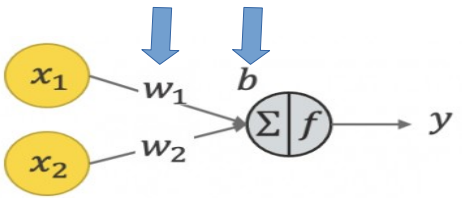
- Fit()
  - Atualização dos pesos e bias com base no erro

```
def fit(X, y, learning_rate=0.001, epochs=100):
      n_features = X.shape[1]

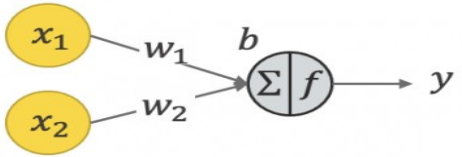
      # Iniacialização
      weights = weights = np.random.rand(n_features)
      bias = 0

      # Iterating until the number of epochs
      for epoch in range(epochs):
          # iteração entre as amostras
          for i in range(len(X)):
              z = np.dot(X, weights) + bias # Produto escalar e bias
              y_pred = activation(z) #Função de ativação
              loss = (y[i] - y_pred[i]) #calculo da perda

              #Atualização dos pesos com base no erro
              weights = weights + learning_rate * loss * X[i]
      return weights, bias. learning_rate * loss
```



Perceptron – Implementação Python



- Fit()
  - Criar o dataset e treinar

```
#Features e Labels
X = np.array([[0, 0, 0],
              [0, 1, 0],
              [1, 1, 0],
              [1, 1, 1]])

y = np.array([1,
              0,
              0,
              1])


w,b = fit(X, y,
          learning_rate=0.001,
          epochs=100)
```

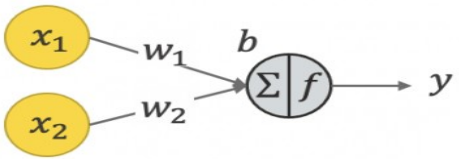
```
def fit(X, y, learning_rate=0.001, epochs=100):
      n_features = X.shape[1]

      # Iniacialização
      weights = weights = np.random.rand(n_features)
      bias = 0

      # Iterating until the number of epochs
      for epoch in range(epochs):
          # iteração entre as amostras
          for i in range(len(X)):
              z = np.dot(X, weights) + bias # Produto escalar e bias
              y_pred = activation(z) #Função de ativação
              loss = (y[i] - y_pred[i]) #calculo da perda

              #Atualização dos pesos com base no erro
              weights = weights + learning_rate * loss * X[i]
              bias = bias + learning_rate * loss
      return weights, bias
```

Perceptron – Implementação Python



- Predict(): Utilizar os pesos do fit e testar

```
#Features e Labels
X = np.array([[0, 0, 0],
              [0, 1, 0],
              [1, 1, 0],
              [1, 1, 1]])

y = np.array([1,
              0,
              0,
              1])


w,b = fit(X, y,
          learning_rate=0.001,
          epochs=100)

pred = predict(X,w,b)

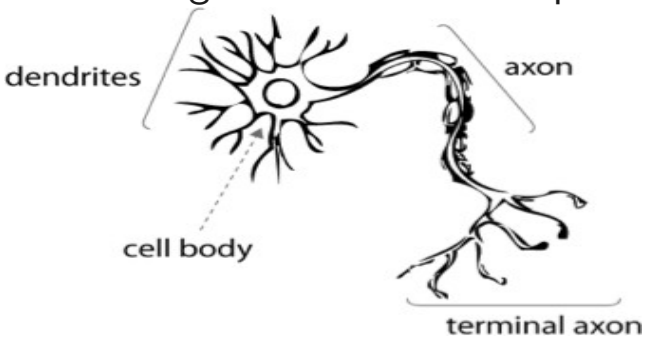
print(pred)
```

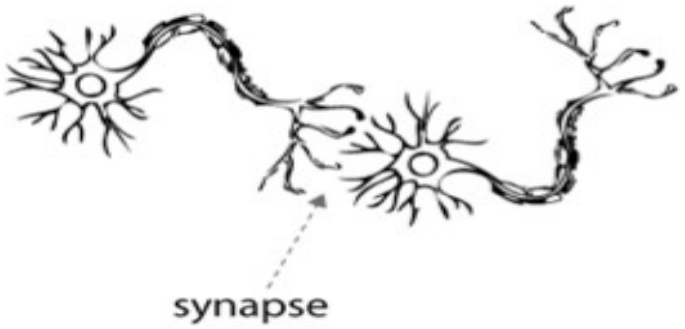
```
def predict(X,weights,bias):
      z = np.dot(X, weights) + bias
      return activation(z)
```

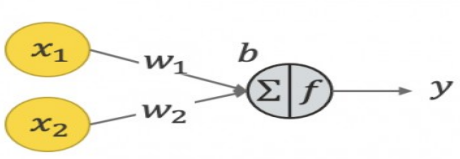
Redes Neurais Artificiais

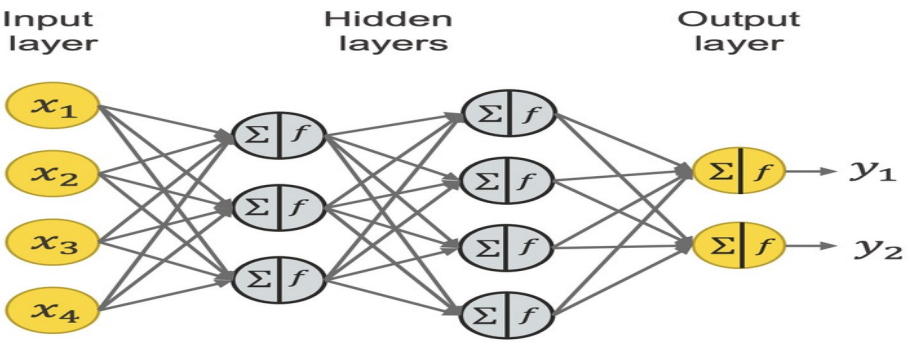
Relembrando

- Modelo Biológico vs Modelo Computacional



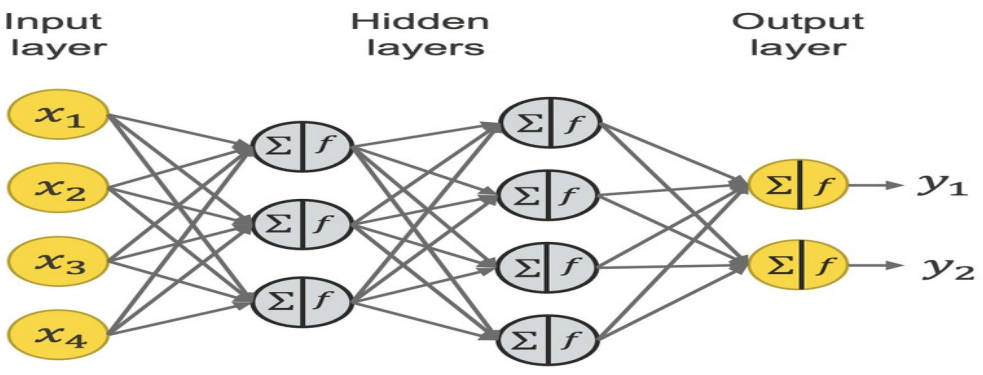






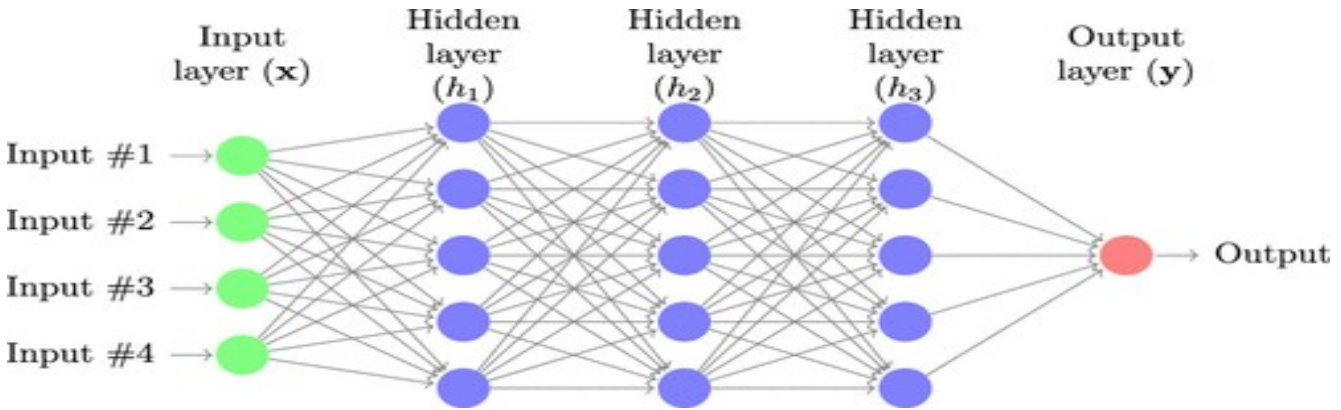
Multi-Layer Perceptron

- Multi-Layer Perceptron (MLP)
  - Neurônios (Perceptrons) interconectados
  - Camada de Entrada (Features)
  - Camada de Saída (Classes)
  - Otimização: Ajuste de pesos



Multi-Layer Perceptron

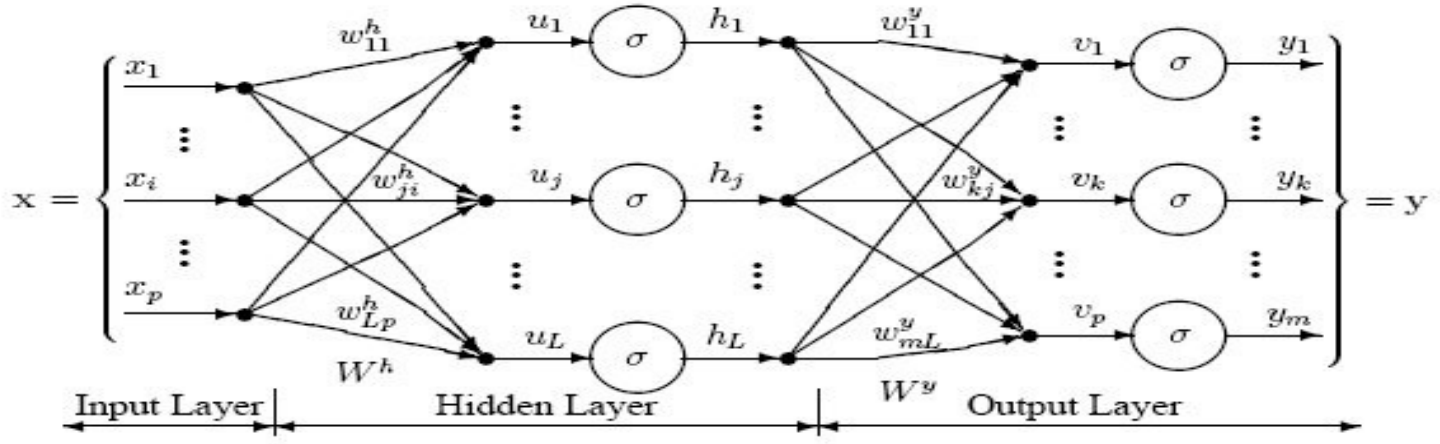
- Multi-Layer Perceptron (MLP)
  - Não há limite de camadas e perceptrons



Multi-Layer Perceptron

Treinamento:

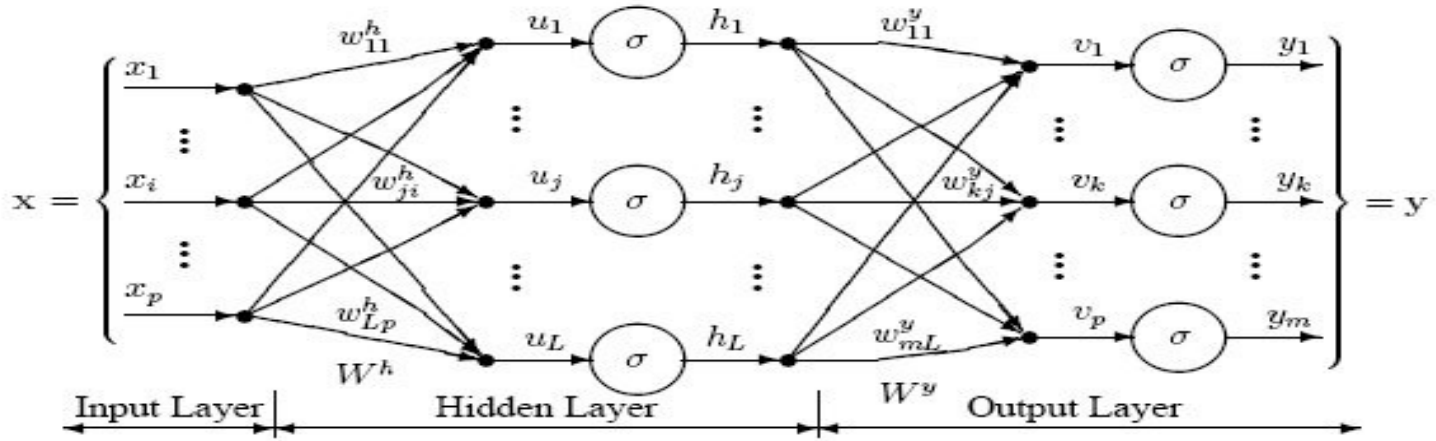
- Feed-Forward: Computa os pesos para o o ‘batch’ de amostras
- Pe



Multi-Layer Perceptron

Treinamento:

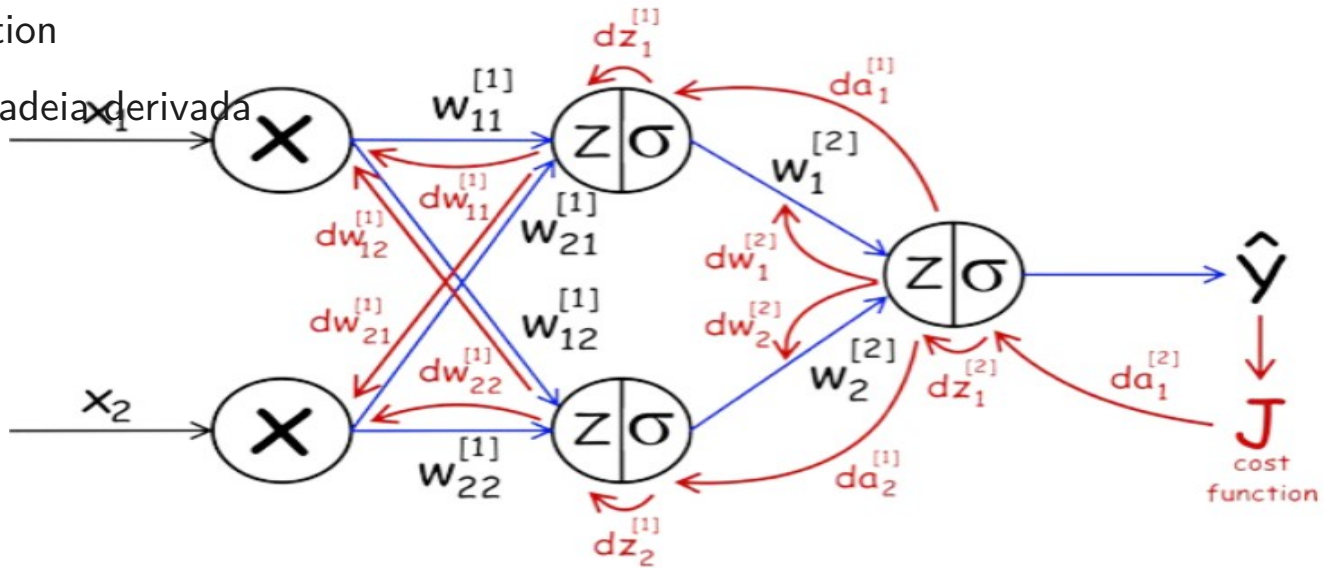
- Pergunta? E como determinar o erro para cada peso na rede?
- Backpropagation
-



Multi-Layer Perceptron

Treinamento:

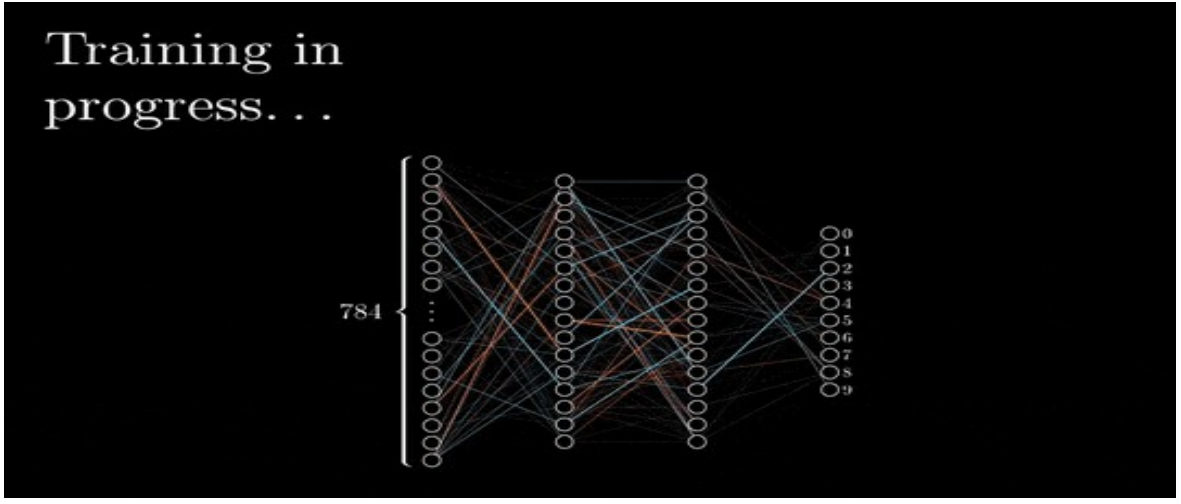
- Pergunta? E como determinar o erro para cada peso na rede?
- Backpropagation
  - Regra da cadeia derivada



Multi-Layer Perceptron

Treinamento:

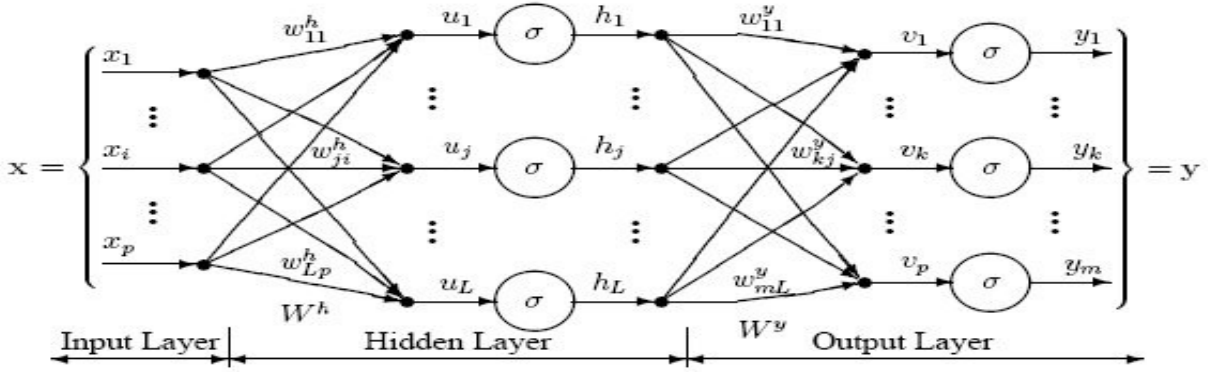
- Pergunta? E como determinar o erro para cada peso na rede?
- Backpropagation
  - Regra da cadeia derivada



Multi-Layer Perceptron

Quais são os parâmetros de uma MLP:

- Neurônios
- Camadas
- Ativação
- Loss
- ……

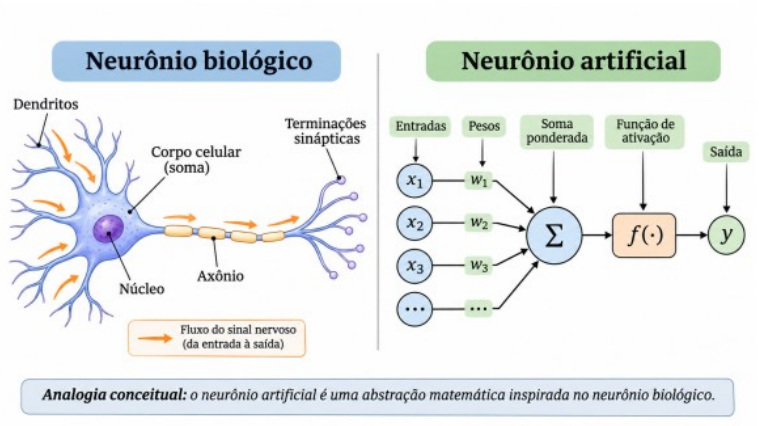


- E como definir tudo isso?
  - Ajuste empírico, com base em experimentação sistemática
  - Separação dos dados em treino, validação e teste para avaliar generalização
  - Uso de técnicas de busca (grid search, random search, Bayesian optimization) para selecionar a melhor configuração

Resumo da Aula

Resumo da Aula

Neurônio Artificial: Origem e Fundamentos



Analogia conceitual: *o neurônio artificial é uma abstração matemática inspirada no neurônio biológico.*

1958 – Frank Rosenblatt

- Desenvolveu o *Perceptron*.
- Introduziu o aprendizado por ajuste dos pesos.
- Classificação de padrões a partir de exemplos.

Atualmente

- Unidade matemática básica das redes neurais artificiais.
- Aplicações em classificação, regressão, visão computacional e IA generativa.

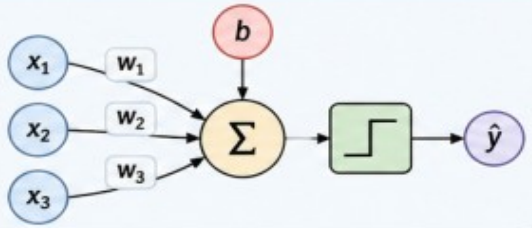
Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6), 386–408.

Como o Perceptron Aprende? Otimização

Conjunto de treino (5 amostras)

Amostra	Idade (norm.)	Pressão (norm.)	Colesterol (norm.)	Risco (y)
1	0,20	0,30	0,40	0
2	0,80	0,90	0,70	1
3	0,60	0,70	0,60	1
4	0,10	0,40	0,30	0
5	0,90	0,80	0,90	1

O Perceptron

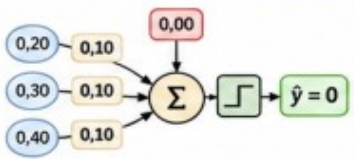


Treinamento

1. Para cada amostra, calcula-se a saída (forward).
2. Se houver erro ($\hat{y} \neq y$), os pesos e o viés são ajustados.
3. Repete-se para todas as amostras.

Iteração 1 – Amostra 1

x = [0,20 0,30 0,40] y = 0



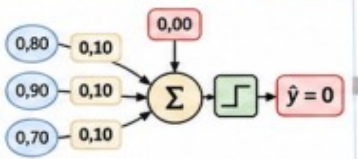
✓ Predição correta (sem ajuste)

Pesos (após iteração 1)

w = [0,10 0,10 0,10] b = 0,00

Iteração 2 – Amostra 2

x = [0,80 0,90 0,70] y = 1



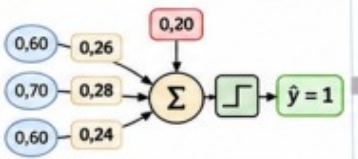
✗ Erro ($\hat{y} = 0, y = 1$) Ajustando os pesos...

Pesos (após iteração 2)

w = [0,26 0,28 0,24] b = 0,20

Iteração 3 – Amostra 3

x = [0,60 0,70 0,60] y = 1



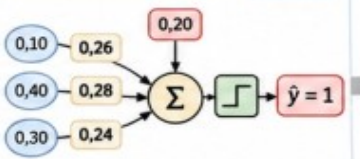
✓ Predição correta (sem ajuste)

Pesos (após iteração 3)

w = [0,26 0,28 0,24] b = 0,20

Iteração 4 – Amostra 4

x = [0,10 0,40 0,30] y = 0



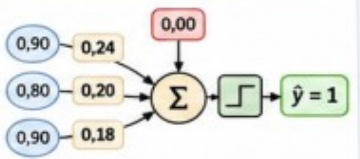
✗ Erro ($\hat{y} = 1, y = 0$) Ajustando os pesos...

Pesos (após iteração 4)

w = [0,24 0,20 0,18] b = 0,00

Iteração 5 – Amostra 5

x = [0,90 0,80 0,30] y = 1



✓ Predição correta (sem ajuste)

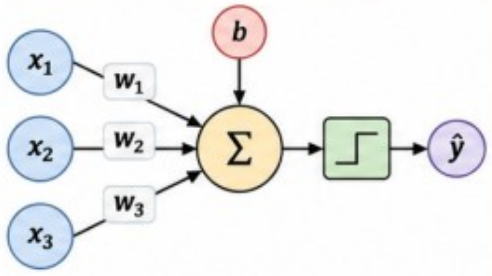
Pesos (final)

w = [0,24 0,20 0,18] b = 0,00

Problema de otimização: ajustar os pesos e o viés para minimizar os erros de classificação no conjunto de treino.

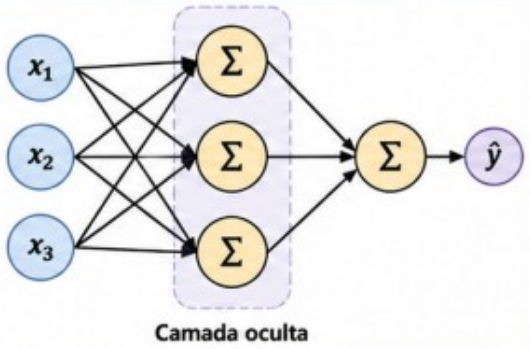
Do Perceptron às Redes Neurais Multicamadas

Perceptron
(um neurônio)



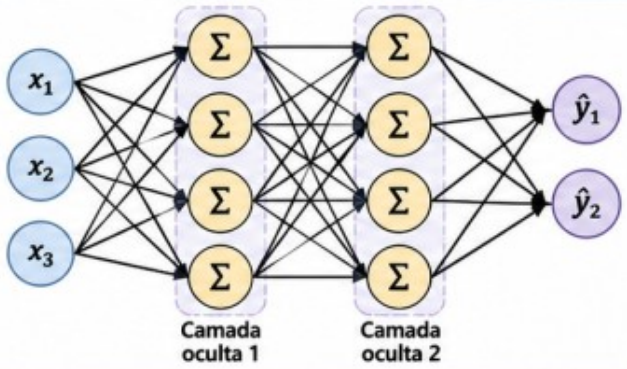
- Combina as entradas com pesos.
- Aplica uma função de ativação.
- Resolve problemas lineares (padrões simples).

Múltiplos Neurônios
(uma camada oculta)



- Vários neurônios aprendem diferentes combinações das entradas.
- A camada oculta cria novas representações dos dados, mais informativas.

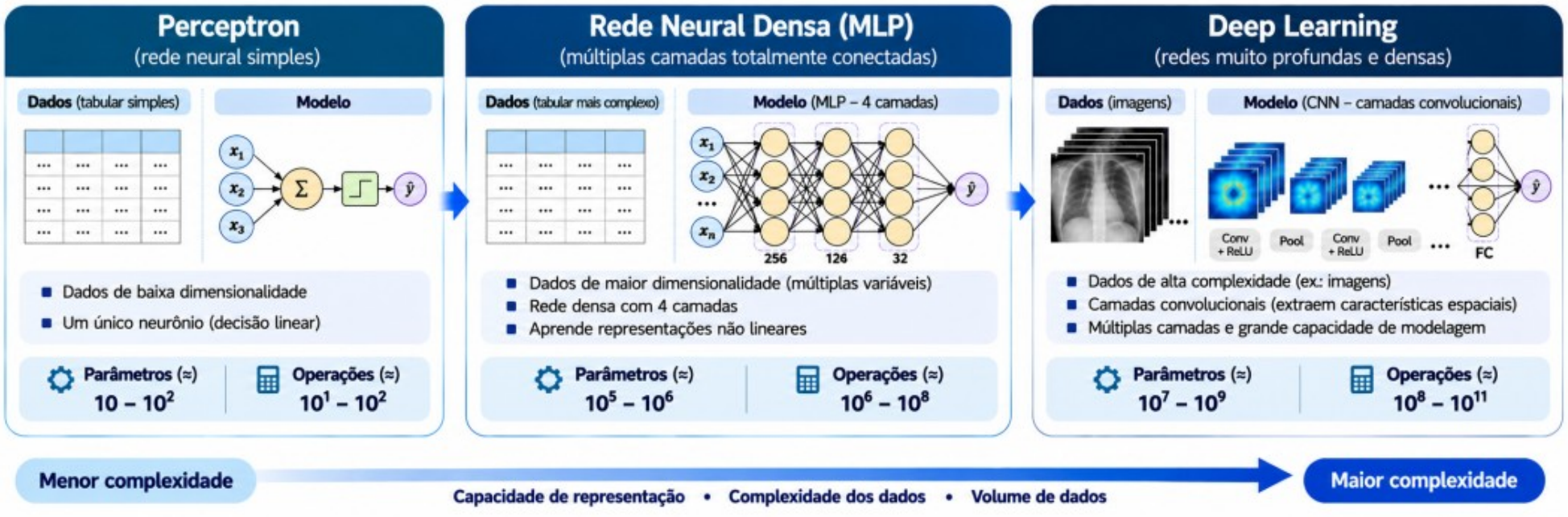
Rede Neural (MLP)
(múltiplas camadas)



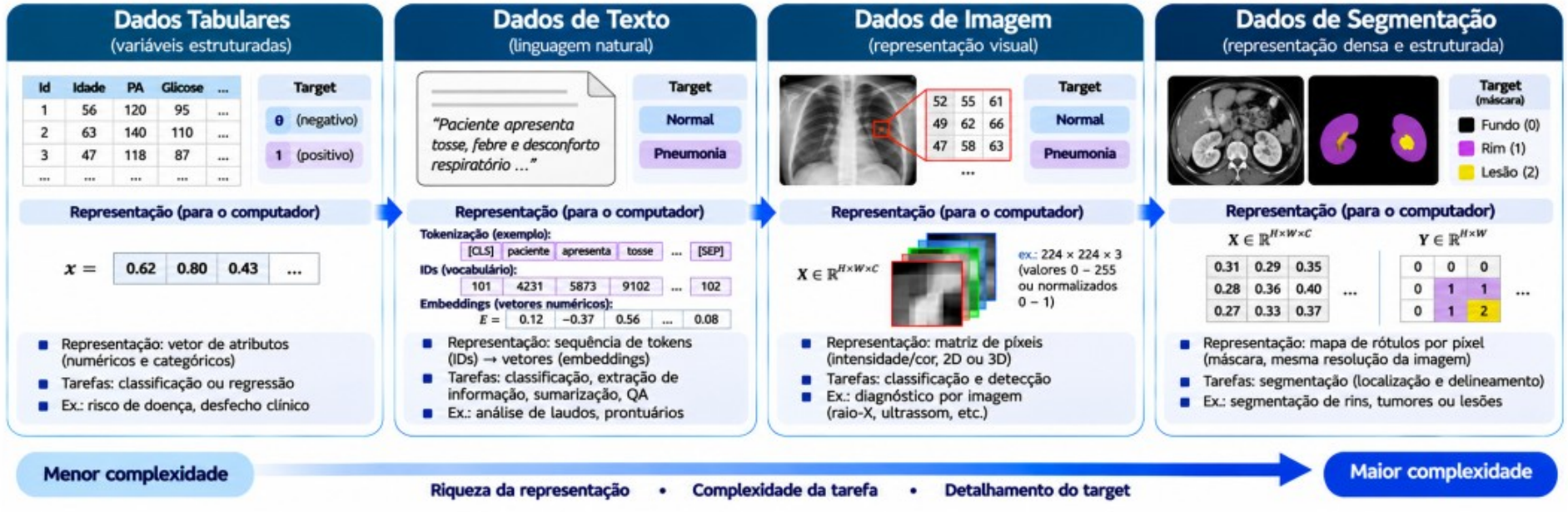
- Múltiplas camadas aprendem representações progressivamente mais complexas.
- Permite modelar relações não lineares e resolver problemas mais desafiadores (ex.: imagens, textos, sinais).

Ideia central: uma rede neural é composta por vários perceptrons organizados em camadas, aprendendo, de forma conjunta, representações cada vez mais complexas dos dados.

Do Perceptron ao Deep Learning: Evolução das Redes Neurais



Evolução das Representações (Dados)



Estado da Arte

